

Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное
Образовательное учреждение высшего образования
МОСКОВСКИЙ ПОЛИТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
(МОСКОВСКИЙ ПОЛИТЕХ)

ДОПУЩЕН К ЗАЩИТЕ

Заведующий кафедрой

_____ / Пухова Е. А. /

Руководитель образовательной программы

_____ / Даньшина М. В. /

ВЫПУСКНАЯ КВАЛИФИКАЦИОННАЯ РАБОТА

по теме:

**ВЕБ И МОБИЛЬНОЕ ПРИЛОЖЕНИЕ ДЛЯ КОРПОРАТИВНОГО
ОБУЧЕНИЯ С ПОДДЕРЖКОЙ АДАПТИВНОГО ТЕСТИРОВАНИЯ**

по направлению 09.03.03 Прикладная информатика

Образовательная программа (профиль) «Корпоративные информационные
системы»

Студент: _____ / Мурадян Лусинэ Гамлетовна, 221–361/
подпись *ФИО, группа*

Руководитель ВКР: _____ / Гонтовой Сергей Викторович, доцент, к.т.н./
подпись *ФИО, уч. звание и степень*

Москва 2026

Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное
Образовательное учреждение высшего образования
МОСКОВСКИЙ ПОЛИТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
(МОСКОВСКИЙ ПОЛИТЕХ)

ЗАДАНИЕ НА ВЫПУСКНУЮ КВАЛИФИКАЦИОННУЮ РАБОТУ
по направлению 09.03.03 Прикладная информатика
Образовательная программа (профиль) «Корпоративные информационные
системы»

Тема ВКР	Веб и мобильное приложение для корпоративного обучения с поддержкой адаптивного тестирования
ПРАКТИЧЕСКИЙ РЕЗУЛЬТАТ	
Назначение	Веб и мобильное приложение для корпоративного обучения с веб-панелью администрирования, мобильным клиентом обучающегося и серверной частью с поддержкой адаптивного тестирования и логической изоляции данных организаций-арендаторов. Назначение — управление учебным контентом, назначениями, прохождением курсов и тестов, формирование рекомендаций по слабым темам и аналитика результатов обучения.
Основные функции	1. Управление пользователями и ролями (обучающийся, преподаватель, администратор организации, системный администратор). 2. Создание и редактирование курсов, уроков и тестов преподавателями. 3. Назначение курсов пользователям и группам администраторами. 4. Прохождение курсов и тестов обучающимися с мобильного приложения. 5. Адаптивное тестирование с подстройкой сложности заданий по результатам. 6. Формирование рекомендаций по повторению материалов. 7. Аналитика и отчёты по прогрессу для администраторов и преподавателей. 8. Поддержка нескольких организаций (multi tenant архитектура).

Используемые технологии и платформы	Серверная часть: Python, FastAPI, SQLAlchemy, Alembic, PostgreSQL/SQLite. Веб-панель: React, TypeScript, Vite. Мобильное приложение: Flutter, Dart. Протоколы взаимодействия: HTTP/HTTPS, JSON. Развёртывание: Docker Compose; эксплуатационный контур — арендованный виртуальный сервер с доменом coursum.online.
ВЫПОЛНЕНИЕ РАБОТЫ	
Решаемые задачи	1. Анализ предметной области корпоративного обучения и исходного состояния процессов. 2. Сравнительный анализ существующих систем управления обучением и определение критериев сопоставления. 3. Формулирование функциональных и нефункциональных требований к системе. 4. Обоснование выбора технологий и платформ для разработки программного комплекса. 5. Проектирование архитектуры системы, модели данных и пользовательских сценариев. 6. Реализация серверной части с программным интерфейсом, веб панели администратора и преподавателя, мобильного приложения обучающегося. 7. Описание алгоритмов адаптивного тестирования, формирования рекомендаций и изоляции данных организаций арендаторов. 8. Проведение испытаний по программе и методике испытаний и фиксация результатов на критическом пути пользователя. 9. Раскрытие аспектов информационной безопасности, надёжности и условий лицензирования. 10. Подготовка приложений по ГОСТ ЕСПД (техническое задание, методика испытаний, руководство пользователя, инструкция по развёртыванию, листинги кода).
Состав технической документации	1. Техническое задание на разработку системы. 2. Программа и методика испытаний. 3. Руководство пользователя (мобильное приложение). 4. Руководство администратора (веб-панель). 5. Описание серверной части и API.

Состав графической части	1. Контейнерная диаграмма (C4) программного комплекса. 2. Диаграмма компонентов серверной части. 3. Диаграмма прецедентов (use case). 4. Инфологическая модель БД. 5. Даталогическая модель БД. 6. Структурная схема клиентской части. 7. Структурная схема серверной части (маршруты, доступ, сервисы, данные). 8. Диаграмма последовательности прохождения адаптивного теста. 9. Схема алгоритма изменения сложности. 10. Схема развёртывания Docker Compose. 11. Скриншоты экранов веб-панели и мобильного приложения – 20 шт. 12. Диаграмма BPMN «AS-IS»
---------------------------------	---

ПЛАН РАБОТЫ НАД ВКР

[illegible]

РУКОВОДИТЕЛЬ ОП:

«__»____2026, _____ / Даньшина Марина Владимировна /
подпись *ФИО, уч. звание и степень*

РУКОВОДИТЕЛЬ ВКР:

«__»____2026, _____ / Гонтовой Сергей Викторович, доцент, к.т. н. /
подпись *ФИО, уч. звание и степень*

СТУДЕНТ:

«__»____2026, _____ / Мурадян Лусинэ Гамлетовна, 221–361/ /
подпись *ФИО, группа*

АННОТАЦИЯ

Наименование работы: «Веб и мобильное приложение для корпоративного обучения с поддержкой адаптивного тестирования».

Цель работы: разработка клиент-серверного программного комплекса корпоративного обучения с адаптивным тестированием и логической изоляцией данных организаций-арендаторов.

Задачи: анализ предметной области и аналогов; формирование требований; проектирование архитектуры и базы данных; реализация серверной части, веб-панели и мобильного приложения; описание алгоритмов адаптивного тестирования; подготовка материалов внедрения и испытаний по ГОСТ ЕСПД.

Объект и предмет исследования: процессы организации корпоративного обучения и методы проектирования клиент-серверных информационных систем с адаптивным тестированием.

Работа содержит введение, три главы, заключение, 55 источников (25 печатных, 30 электронных) и пять приложений. Объём — 73 страницы, включая 14 рисунков и 12 таблиц.

Во Введении обоснована актуальность, сформулированы цель и задачи, определены научная новизна и практическая значимость.

В Первой главе — анализ предметной области, обзор и сравнение LMS-аналогов, выбор технологического стека.

Во Второй главе — архитектура системы, модель данных, алгоритмы адаптивного тестирования, реализация серверной и клиентской частей.

В Третьей главе — внедрение, тестирование, информационная безопасность и эксплуатация.

В Заключении — выводы и направления развития.

Результаты работы апробированы развертыванием программы на виртуальном выделенном сервере и публикации исходного кода на GitHub.

СОДЕРЖАНИЕ

ВВЕДЕНИЕ	10
1 ПРЕДМЕТНАЯ ОБЛАСТЬ И ТЕХНОЛОГИИ	14
1.1 Предметная область корпоративного обучения и исходное состояние	14
1.2 Постановка проблемы и назначение системы	16
1.3 Требования к системе	19
1.4 Анализ аналогов	24
1.5 Выбор технологий и платформ	26
1.6 Выводы по главе	32
2 ТЕХНИЧЕСКАЯ РЕАЛИЗАЦИЯ СИСТЕМЫ	33
2.1 Архитектура системы	33
2.2 Сценарии использования	35
2.3 Модель данных	38
2.4 Проектирование интерфейсов	41
2.5 Реализация web- и mobile-клиентов	44
2.6 Реализация серверной логики и алгоритмов	48
2.7 Выводы по главе	53
3 ВНЕДРЕНИЕ И ЭКСПЛУАТАЦИЯ ПРОГРАММНОГО КОМПЛЕКСА	54
3.1 Анализ внедрения и оценка результатов реализации	54
3.2 Тестирование программного комплекса	57
3.3 Аспекты информационной безопасности	59
3.4 Надёжность, отказоустойчивость и производительность	61
3.5 эксплуатация и развитие	62
3.6 Выводы по главе	62
ЗАКЛЮЧЕНИЕ	64
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	66
ПРИЛОЖЕНИЕ А. ТЕХНИЧЕСКОЕ ЗАДАНИЕ	72

ПРИЛОЖЕНИЕ Б. ПРОГРАММА И МЕТОДИКА ИСПЫТАНИЙ	77
ПРИЛОЖЕНИЕ В. РУКОВОДСТВО ПОЛЬЗОВАТЕЛЯ МОБИЛЬНОГО ПРИЛОЖЕНИЯ И РУКОВОДСТВО АДМИНИСТРАТОРА ВЕБ-ПАНЕЛИ	80
ПРИЛОЖЕНИЕ Г. РУКОВОДСТВО ПО РАЗВЁРТЫВАНИЮ ПРОГРАММНОГО КОМПЛЕКСА	84
ПРИЛОЖЕНИЕ Д. ЛИСТИНГИ КЛЮЧЕВОГО КОДА	88

ВВЕДЕНИЕ

Выпускная квалификационная работа посвящена разработке корпоративной системы управления обучением, предназначенной для организации учебного контента, назначения курсов, контроля результатов и проведения адаптивного тестирования в рамках единой клиент–серверной платформы[1].

Актуальность темы обусловлена тем, что в корпоративной среде обучение сотрудников всё чаще строится как непрерывный процесс, включающий адаптацию новых сотрудников (онбординг), обязательные программы, проверку знаний и повторение проблемных тем. При использовании разрозненных инструментов усложняется управление пользователями и ролями, затрудняется назначение курсов, отсутствует единый контур аналитики и слабо поддерживается персонализация учебной траектории. Дополнительным фактором является распространение мобильного обучения, при котором слушатели проходят курсы короткими сессиями со смартфона и ожидают простой навигации, быстрого доступа к материалам и прозрачной обратной связи по результатам тестирования[2, 3].

Для корпоративных сценариев также существенна возможность обслуживания нескольких организаций в рамках одной платформы с логическим разделением данных, ролей и учебного контента. Такой подход позволяет использовать единое программное решение для нескольких арендаторов, сохраняя изоляцию данных и независимость администрирования[4, 5].

Цель работы — разработать корпоративную LMS с веб-панелью администрирования и мобильным клиентом слушателя, обеспечивающую управление учебным контентом, назначениями, адаптивным тестированием и аналитикой результатов обучения. Система адресована ролям системного администратора платформы, администратора организации, преподавателя и слушателя.

Для достижения поставленной цели необходимо решить следующие задачи:

- 1) проанализировать предметную область корпоративного обучения и исходное состояние процессов;
- 2) выявить основные проблемы организации учебного процесса и обосновать необходимость автоматизации;
- 3) сформулировать функциональные и нефункциональные требования к системе;
- 4) выполнить сравнительный анализ существующих LMS-решений и определить критерии сопоставления;
- 5) обосновать выбор технологий и платформ для разработки программного комплекса;
- 6) спроектировать архитектуру системы, модель данных и пользовательские сценарии;
- 7) реализовать серверную, веб- и мобильную части системы;
- 8) описать алгоритмы адаптивного тестирования, формирования рекомендаций и изоляции данных арендаторов;
- 9) описать процедуры опытного ввода в эксплуатацию и развёртывания стенда (приложение Г);
- 10) провести испытания по программе и методике испытаний и зафиксировать ключевые результаты на критическом пути пользователей (приложение Б);
- 11) раскрыть аспекты информационной безопасности, надёжности и ограничений одного экземпляра;
- 12) сформулировать выводы и подготовить приложения по ГОСТ ЕСПД (техническое задание, методика испытаний, руководство пользователя, инструкция по развёртыванию, листинги кода).

Объектом исследования являются процессы организации корпоративного обучения, контроля прохождения курсов и оценки результатов слушателей в цифровой образовательной среде.

Предметом исследования являются методы и средства проектирования и разработки клиент–серверной информационной системы корпоративного обучения, включающей веб-интерфейс администрирования, мобильный клиент слушателя, адаптивное тестирование и механизмы аналитики.

Методы исследования: системный анализ процессов корпоративного обучения; сравнительный анализ программных аналогов по формализованным критериям и весам; объектно-ориентированное проектирование архитектуры и базы данных; документирование требований и трассировка к тест-кейсам; контроль качества по программе и методике испытаний (приложение Б)[6, 7, 8].

Информационная база исследования: нормативные документы по оформлению отчётной и программной документации[9, 10, 11, 12, 13, 14, 15, 16, 17]; методические рекомендации вуза по ВКР[1]; учебная и специальная литература по информационным системам, базам данных и тестированию[18, 19, 20, 21, 22, 23, 24, 25]; официальная документация использованных платформ[26, 27, 28, 29, 30].

Принятый вариант архитектуры — модульный монолит на стороне сервера с отдельными клиентами и логической изоляцией данных арендаторов в общей СУБД — обоснован в главе 2 и подтверждён опытным развёртыванием и испытаниями (приложение Б).

Практическим результатом является программный комплекс: серверное приложение на FastAPI, веб-панель на React и TypeScript, мобильное приложение слушателя на Flutter; роли, аутентификация, управление курсами и тестами, адаптивное тестирование и рекомендации. Исходные тексты и порядок сборки приведены в открытом репозитории <https://github.com/lunahobi/coursum>.

Значимость результата — возможность вести корпоративное обучение в едином контуре от контента до аналитики и повторного обучения по слабым темам.

Работа содержит введение, три главы, заключение, 55 наименований в списке источников и пять приложений по ГОСТ[1, 8]; иллюстративный материал и объём уточняются при верстке PDF.

1 ПРЕДМЕТНАЯ ОБЛАСТЬ И ТЕХНОЛОГИИ

1.1 Предметная область корпоративного обучения и исходное состояние

Системы управления обучением применяются для централизованного хранения учебных материалов, назначения курсов, контроля прохождения и фиксации результатов. В корпоративной среде LMS также применяют для адаптации новых сотрудников (онбординг), обязательного обучения, внутренних регламентов, курсов по информационной безопасности и повышения квалификации сотрудников[31, 32].

Для рассматриваемой предметной области характерно наличие нескольких устойчивых ролей. Системный администратор поддерживает общесистемные настройки и контролирует работу платформы. Администратор организации управляет пользователями, ролями, курсами и назначениями в пределах своей организации. Преподаватель создаёт учебный контент, тесты и анализирует результаты. Слушатель проходит назначенные курсы, изучает уроки, выполняет тесты и получает рекомендации по слабым темам.

В исходном состоянии процесс обучения часто строится с использованием нескольких несвязанных инструментов: документы и видео хранятся отдельно, списки обучающихся ведутся вручную, результаты тестирования не агрегируются, а повторение слабых тем организуется без формальной аналитики. Это приводит к ряду практических проблем:

- 1) отсутствует единый источник данных о пользователях, курсах и результатах;
- 2) повышаются трудозатраты на назначение курсов и контроль прохождения;
- 3) преподаватель не получает целостной картины по ошибкам и слабым темам;

4) слушатель проходит одинаковые по сложности тесты вне зависимости от уровня подготовки;

5) использование мобильных устройств поддерживается ограниченно или неудобно.

На рисунке 1 представлена схема процесса корпоративного обучения в цифровой среде. Диаграмма показывает последовательность действий от подготовки контента и назначения курса до прохождения тестирования и анализа результатов.

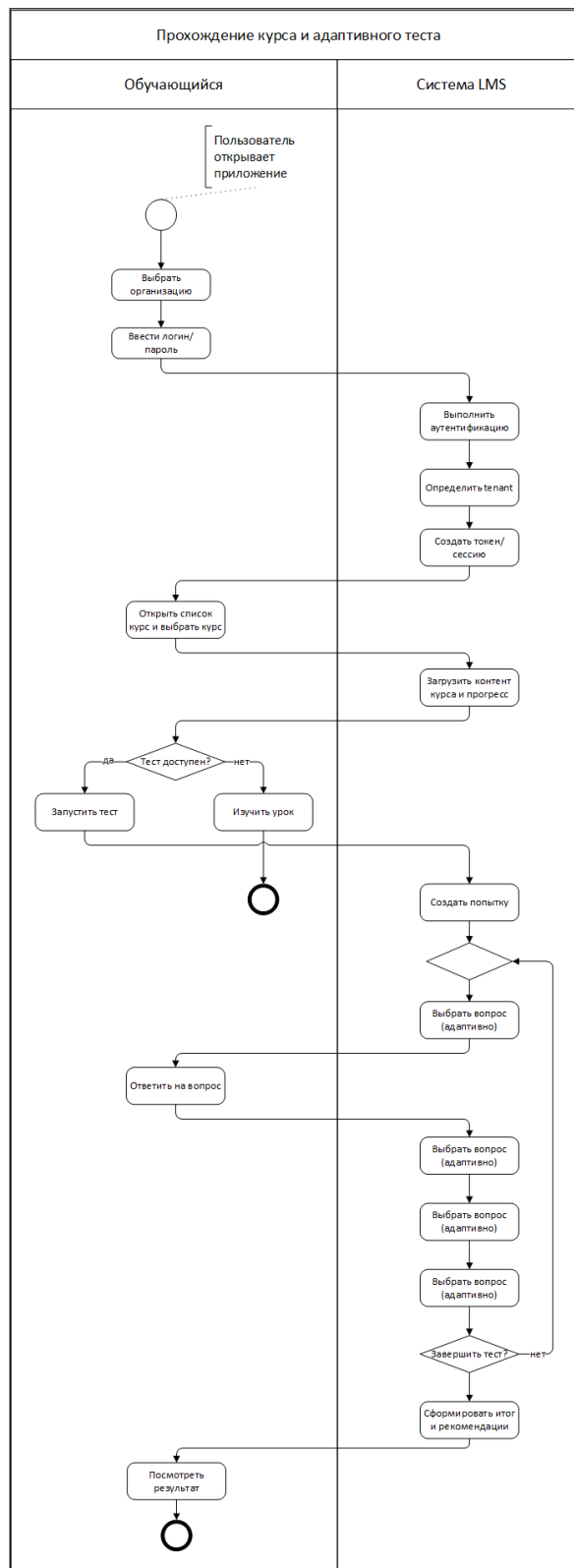


Рисунок 1 – Схема процесса корпоративного обучения в цифровой среде

1.2 Постановка проблемы и назначение системы

Проблема разработки состоит в необходимости объединить в одной системе функции администрирования, хранения контента, назначения учеб-

ных программ, мобильного доступа, тестирования и анализа результатов без усложнения пользовательских сценариев. Типовые LMS нередко обладают избыточным или трудно настраиваемым функционалом, а их мобильный пользовательский опыт не всегда соответствует сценарию коротких учебных сессий[33, 3].

Назначение разрабатываемой системы заключается в поддержке полного цикла корпоративного обучения:

- 1) создание и редактирование пользователей, ролей, курсов, уроков и тестов;
- 2) назначение курсов слушателям;
- 3) прохождение уроков и фиксирование прогресса;
- 4) запуск адаптивного теста и подбор вопросов с учётом текущей сложности;
- 5) формирование рекомендаций по слабым темам;
- 6) просмотр аналитики по обучению в административном интерфейсе.

Система должна обеспечивать работу нескольких организаций в одном развёртывании. В проекте эта задача решается не путём создания отдельной базы данных на каждого арендатора, а через логическую мультиарендную модель (несколько организаций в одном экземпляре): идентификация организации в запросе, проверка участия пользователя в организации и фильтрация данных по полю `tenant_id`. Такой подход достаточен для учебного минимально жизнеспособного продукта (MVP) и соответствует текущей реализации проекта.

На рисунке 2 показана диаграмма вариантов использования, отражающая ключевые действия ролей в системе. Диаграмма фиксирует распределение функций между системным администратором, администратором организации, преподавателем и слушателем.

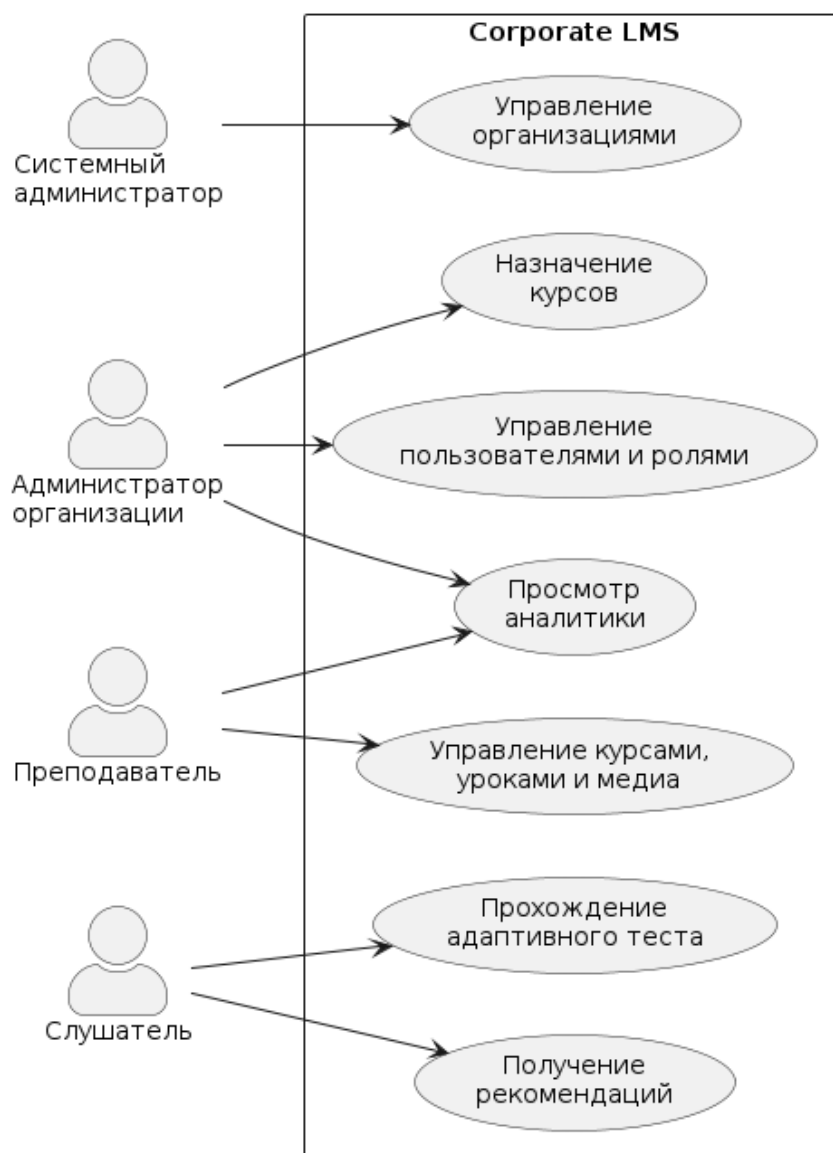


Рисунок 2 – Диаграмма вариантов использования системы

Для более формального представления сценариев в таблице 1 приведены основные прецеденты системы, их акторы и ожидаемые результаты выполнения.

Таблица 1 – Ключевые прецеденты системы

Прецедент	Основной актер	Результат выполнения
Управление организациями	Системный администратор	Созданы и настроены организации, доступные для дальнейшей работы пользователей.
Управление пользователями и ролями	Администратор организации	В организации созданы пользователи и назначены роли в соответствии с полномочиями.
Управление курсами, уроками и медиа	Преподаватель	Подготовлен учебный контент, доступный для назначения слушателям.
Назначение курса	Администратор организации	Выбранный курс назначен слушателю и появился в его списке обучения.
Прохождение адаптивного теста	Слушатель	Сформирована завершённая попытка с результатом и историей ответов.
Получение рекомендаций	Слушатель	Пользователь получает перечень тем и материалов для повторения.
Просмотр аналитики	Преподаватель, администратор организации	Получены агрегированные показатели по курсам, попыткам и успеваемости.

1.3 Требования к системе

Требования к системе сформированы с учётом методических рекомендаций по ВКР и подхода к анализу требований, описанного в учебнике С. В. Куликова. Под требованием в работе понимается проверяемое описание того, какие функции должна выполнять система и при каких условиях она должна работать, чтобы решать задачу пользователя и заказчика[1, 7]. Поэтому требования далее разделены на функциональные и нефункциональные: первые фиксируют ожидаемое поведение программного комплекса, вторые задают ограничения на безопасность, изоляцию данных, архитектуру, интерфейсы и эксплуатационные свойства.

Бизнес-цель системы заключается в создании единого программного комплекса для корпоративного обучения, который позволяет управлять пользователями, курсами, назначениями, прохождением уроков, адаптивным тестированием и аналитикой без использования набора разрозненных инструментов. Основными ролями являются системный администратор, администратор организации, преподаватель и слушатель. Требования сформулированы так, чтобы каждое из них было атомарным, непротиворечивым, недвусмысленным и проверяемым в ходе испытаний[7].

1.3.1 Функциональные требования

Функциональные требования описывают действия, которые система должна предоставлять пользователям и клиентским приложениям.

1) FR-01 – система должна обеспечивать регистрацию и аутентификацию пользователей.

2) FR-02 – система должна выдавать токены доступа и обновления для последующей работы с защищёнными функциями.

3) FR-03 – система должна определять активную организацию пользователя по контексту запроса.

4) FR-04 – система должна поддерживать роли слушателя, преподавателя, администратора организации и системного администратора.

5) FR-05 – администратор организации должен иметь возможность создавать пользователей и изменять состояние их доступа.

6) FR-06 – администратор организации должен иметь возможность назначать пользователям роли в пределах своей организации.

7) FR-07 – преподаватель или администратор должен иметь возможность создавать и редактировать курсы.

8) FR-08 – преподаватель или администратор должен иметь возможность создавать и редактировать уроки с текстовым и медийным содержанием.

9) FR-09 – преподаватель или администратор должен иметь возможность создавать тесты, вопросы, варианты ответов и связи вопросов с темами.

10) FR-10 – администратор организации должен иметь возможность назначать курс слушателю.

11) FR-11 – слушатель должен иметь возможность просматривать назначенные курсы, открывать уроки и фиксировать прогресс прохождения.

12) FR-12 – слушатель должен иметь возможность начать попытку тестирования, получать вопросы, отправлять ответы и завершать попытку.

13) FR-13 – после каждого ответа система должна изменять текущую сложность адаптивной попытки не более чем на один уровень.

14) FR-14 – система должна выбирать следующий вопрос среди ещё не заданных вопросов с учётом текущей сложности.

15) FR-15 – после завершения попытки система должна сохранять результат, историю ответов и выявленные слабые темы.

16) FR-16 – система должна формировать рекомендации по повторению материалов на основании слабых тем.

17) FR-17 – слушатель должен иметь возможность просматривать результат тестирования и рекомендации.

18) FR-18 – преподаватель и администратор организации должны иметь возможность просматривать аналитику по пользователям, курсам, тестам и назначенным обучением.

1.3.2 Нефункциональные требования

Нефункциональные требования фиксируют свойства качества и ограничения, которые должны учитываться при проектировании, реализации и проверке системы.

1) NFR-01 – доступ к защищённым операциям должен выполняться только после аутентификации и проверки роли пользователя,

2) NFR-02 – пароли пользователей должны храниться только в виде криптографических хэшей,

3) NFR-03 – ключевые действия пользователей должны фиксироваться в журнале аудита,

4) NFR-04 – данные разных организаций не должны смешиваться при выполнении пользовательских запросов,

5) NFR-05 – каждая сущность, зависящая от организации, должна быть связана с идентификатором организации,

6) NFR-06 – серверная часть должна предоставлять REST API с обменом данными в формате JSON,

7) NFR-07 – система должна поддерживать воспроизводимое контейнеризированное развёртывание для демонстрационного стенда,

8) NFR-08 – серверная часть должна быть разделена на слои и подсистемы, чтобы упростить развитие и проверку поведения,

9) NFR-09 – веб-интерфейс должен быть ориентирован на административные сценарии управления пользователями, курсами, тестами и аналитикой,

10) NFR-10 – мобильный интерфейс должен быть ориентирован на сценарии слушателя: вход, просмотр курсов, прохождение уроков, тестирование и рекомендации,

11) NFR-11 – пользовательский интерфейс должен поддерживать русский и английский языки,

12) NFR-12 – отклик основных интерактивных операций на демонстрационных данных должен быть приемлемым для работы пользователя без ощущения зависания,

13) NFR-13 – система должна использовать общую базу данных с логической изоляцией данных организаций,

14) NFR-14 – система должна хранить курсы, уроки, вопросы, попытки, результаты, слабые темы и рекомендации в структурированном виде.

1.3.3 Свойства качественных требований

В таблице 2 показано, как сформулированные требования соотносятся с признаками качественных требований, описанными в учебной литературе по тестированию[7].

Таблица 2 – Проверка свойств требований

Свойство	Как обеспечено в работе
Завершённость	Требования охватывают основные роли, пользовательские сценарии, данные, безопасность, изоляцию организаций и клиентские приложения.
Атомарность	Каждое требование описывает одно проверяемое свойство или одну пользовательскую возможность.
Непротиворечивость	Функциональные требования не конфликтуют с ограничениями архитектуры и мультиарендной модели данных.
Недвусмысленность	Требования сформулированы через роли, действия пользователя и наблюдаемый результат.
Проверяемость	Для требований могут быть подготовлены тест-кейсы, а критический путь проверяется в программе и методике испытаний приложения Б.

1.3.4 Сценарий тестирования требований по критическому пути пользователей

Для проверки согласованности требований используется критический путь, проходящий через основные роли системы: администратор организации создаёт пользователя и назначает курс, преподаватель подготавливает курс и тест, слушатель проходит уроки и адаптивное тестирование, после чего преподаватель или администратор просматривает результаты. Такой сценарий позволяет связать требования с будущей реализацией, проектной моделью и испытаниями.

Таблица 3 – Трассировка требований по критическому пути

Группа требований	Проверяемый пользовательский сценарий	Связанные требования
Пользователи и роли	Создание пользователя, вход в систему, выбор организации и проверка доступных функций	FR-01–FR-06, NFR-01–NFR-05
Курсы и уроки	Создание курса, добавление урока и назначение курса слушателю	FR-07–FR-11, NFR-09, NFR-10, NFR-14
Адаптивное тестирование	Запуск попытки, получение вопросов, отправка ответов, изменение сложности и завершение попытки	FR-12–FR-15, NFR-06, NFR-12
Рекомендации и аналитика	Формирование рекомендаций, просмотр результата слушателем и аналитики преподавателем	FR-16–FR-18, NFR-08, NFR-14
Развёртывание	Запуск демонстрационного стенда и проверка доступности серверной, веб- и мобильной частей	NFR-06–NFR-13

1.4 Анализ аналогов

Для сравнения выбраны решения, представляющие разные классы LMS: открытые платформы (*Moodle*, *Open LMS*), облачный вариант на базе Moodle (*MoodleCloud*), платформы массовых онлайн-курсов и коммерческие корпоративные LMS[31, 32, 34, 35]. В отраслевых обзорах подчёркивается значение адаптивных и интеллектуальных подходов к персонализации учебной траектории при массовом цифровом обучении[36, 37].

Сравнение выполняется по критериям, значимым для данной ВКР:

- 1) наличие административных ролей и разделения прав доступа;
- 2) удобство мобильного доступа для слушателя;
- 3) наличие или отсутствие адаптивного тестирования;
- 4) возможности аналитики и работы со слабыми темами;
- 5) поддержка мультиарендного сценария (несколько организаций в одном развёртывании);
- 6) пригодность для учебной разработки и доработки архитектуры.

Для формализации сравнения в таблице 4 заданы критерии и удельные веса w_i такие, что $\sum_i w_i = 1$. Баллы x_{ij} по 10-балльной шкале при необходимости фиксируются по экспертным анкетам критериев главы 1 или по материалам официальной документации аналогов и сведены в таблице 5. Итоговый показатель для варианта j вычисляется как взвешенная сумма[8]

$$S_j = \sum_i w_i x_{ij}. \quad (1)$$

Результаты свёртки приведены в таблице 6. Наивысший показатель у собственной разработки, что согласуется с целью ВКР — продемонстрировать архитектуру, модель данных и алгоритмы в открытом для анализа коде; коммерческие LMS (в таблице представлены iSpring Learn и GetCourse как ориентиры сегмента) получают высокие баллы по готовому функционалу, но

низкие по критерию прозрачности архитектуры для учебного проекта. Численные оценки x_{ij} — ориентировочные и подлежат уточнению по данным экспертной оценки критериев сравнения.

Таблица 4 – Критерии сравнения LMS-аналогов и удельные веса

Критерий i	Содержание	Вес w_i
1	Административные роли и RBAC	0,15
2	Мобильный доступ для слушателя	0,20
3	Адаптивное / персонализированное тестирование	0,25
4	Аналитика и слабые темы	0,15
5	Мультиарендность	0,15
6	Пригодность для анализа архитектуры в ВКР	0,10
Сумма весов		1,00

Таблица 5 – Оценка аналогов по критериям (шкала 1–10, x_{ij})

Критерий	Moodle	Open LMS	MoodleCloud	iSpring	GetCourse	Собственная
1	8	8	7	8	8	8
2	6	6	6	8	8	8
3	5	5	5	7	7	9
4	6	6	6	8	8	8
5	5	5	5	7	7	8
6	4	4	4	4	4	9

Таблица 6 – Свёртка оценок по аналогам (итоговый балл S_j по формуле 1)

Решение j	S_j (условные единицы)
Moodle	5,70
Open LMS	5,70
MoodleCloud	5,55
iSpring Learn (ориентир)	7,20
GetCourse (ориентир)	7,20
Собственная разработка	8,35

Поскольку административные роли поддерживаются всеми рассматриваемыми решениями, в таблице 7 дополнительно приведено качественное сопоставление по ключевым признакам, по которым платформы различаются наиболее заметно для пользователя.

Таблица 7 – Сравнение аналогов по ключевым критериям

Решение	Мобильный доступ	Адаптивное тестирование	Аналитика	Мультиарендность
Moodle	средне	ограничено	да	ограничено
Open LMS	средне	ограничено	да	ограничено
MoodleCloud	средне	ограничено	да	ограничено
Коммерческие LMS	хорошо	частично или да	хорошо	да
Собственная система	хорошо	да	да	да

По результатам сравнения можно сделать следующие выводы. Moodle и производные от него платформы обеспечивают широкий базовый функционал, однако требуют более сложной настройки и не ориентированы на демонстрацию собственной архитектуры в рамках учебной разработки. Коммерческие LMS предлагают зрелые средства мобильного доступа и аналитики, но являются закрытыми и хуже подходят как основа выпускного проекта. Разрабатываемая система уступает крупным платформам по степени зрелости, но лучше соответствует целям ВКР, поскольку позволяет явно продемонстрировать архитектуру, модель данных, адаптивное тестирование и изоляцию данных организаций-арендаторов.

Результаты сравнения показывают, что существующие решения закрывают базовые задачи LMS, однако либо обладают избыточностью и сложностью настройки, либо не ориентированы на исследование и реализацию собственной архитектуры. Для целей ВКР целесообразно разрабатывать собственный программный комплекс, в котором можно явно продемонстрировать клиент–серверную архитектуру, модель данных, алгоритм адаптивного тестирования и изоляцию данных арендаторов.

1.5 Выбор технологий и платформ

Выбор технологий определялся не только их распространённостью, но и соответствием требованиям проекта. Разрабатываемая система должна поддерживать отдельные web- и mobile-клиенты, REST API, изоляцию данных организаций-арендаторов, развитие модели предметной области и воспроизводимое локальное развёртывание. Поэтому для каждой подсистемы рассматривались несколько вариантов, после чего выбиралось решение, лучше всего соответствующее целям ВКР и текущему масштабу проекта.

1.5.1 Критерии выбора технологий

При выборе технологического стека учитывались следующие критерии:

- 1) соответствие клиент–серверной архитектуре с отдельными web- и mobile-клиентами;
- 2) удобство реализации REST API, аутентификации, разграничения доступа и контекста организации-арендатора;
- 3) возможность формального описания модели данных изменений схемы БД;
- 4) пригодность для минимально жизнеспособного продукта (MVP), который необходимо не только разработать, но и продемонстрировать на защите;
- 5) воспроизводимость локального развёртывания без сложной инфраструктуры.

1.5.2 Сравнение технологий и результат выбора

Сравнение основных вариантов приведено в таблице 8. В таблицу включены именно те подсистемы, которые определяют архитектурный облик проекта и влияют на удобство дальнейшей доработки.

Таблица 8 – Сравнение технологий и результаты выбора

Подсистема	Рассмотренные варианты	Выбранная технология и причина
Серверная часть	FastAPI, Django REST Framework, Flask	Выбран FastAPI, так как проект имеет формат API-first и не требует серверной генерации HTML. По сравнению с Django REST Framework решение получается легче и проще для MVP, а по сравнению с Flask требует меньше ручной сборки маршрутов, схем и документации.
Доступ к данным	SQLAlchemy + Alembic, Django ORM, прямой SQL	Выбраны SQLAlchemy и Alembic, поскольку они не привязаны к конкретному веб-фреймворку, позволяют явно описывать модели и контролировать миграции. Прямой SQL усложнил бы развитие, а Django ORM логичнее использовать только вместе с Django.
Web-клиент	React + TypeScript, Vue, Angular	Выбраны React и TypeScript. Для административной панели важны компонентный подход, маршрутизация, работа с формами и типизация данных. Angular для MVP избыточен, а Vue также подходит, но в текущем проекте более удобным оказался гибкий компонентный подход React.
Мобильный клиент	Flutter, React Native, native Android	Выбран Flutter, поскольку он позволяет поддерживать единый код мобильного приложения и быстро собрать интерфейс слушателя. Нативная разработка увеличила бы трудоёмкость, а React Native не давал принципиального преимущества при уже разделённых web- и mobile-сценариях.
Хранение данных	SQLite, PostgreSQL, MySQL	Используется комбинированный подход: SQLite удобен для локальной разработки и тестирования без отдельного сервера, а PostgreSQL лучше подходит для контейнеризованного запуска и демонстрации многопользовательской системы.
Развёртывание	ручной запуск сервисов, Docker Compose, Kubernetes	Выбран Docker Compose, так как он достаточен для MVP и позволяет поднимать серверную часть, web и базу данных одной конфигурацией. Ручной запуск хуже воспроизводим, а Kubernetes для учебного проекта избыточен.

1.5.3 Обоснование выбора серверной части

Для серверной части рассматривались три основных подхода: использование полноценного фреймворка Django с Django REST Framework, минималистичного Flask и API-ориентированного FastAPI. В рамках данной ВКР сервер выполняет роль централизованного REST API для двух клиентов и не требует шаблонной серверной генерации страниц. По этой причине возмож-

ности Django, связанные с административной панелью, шаблонами и встроенной ORM, для проекта оказались избыточными. Flask, напротив, предоставляет слишком низкий уровень абстракции, из-за чего значительную часть инфраструктурного кода пришлось бы собирать вручную[26, 38].

FastAPI оказался наиболее сбалансированным вариантом. Он хорошо подходит для явного описания маршрутов, схем запросов и ответов, зависимостей и проверки входных данных. Это соответствует фактической реализации серверной части, где сервер разделён на группы маршрутов auth, tenants, users, courses, media, tests, recommendations и analytics. Такая структура хорошо сочетается с модульным монолитом и позволяет расширять систему без переработки базовой архитектуры[26].

Для доступа к данным выбраны SQLAlchemy и Alembic[30]. Такой набор позволяет описывать сущности предметной области на уровне моделей, а изменения схемы фиксировать через миграции. Для ВКР это важно, поскольку система включает достаточно большую группу взаимосвязанных сущностей: организации, пользователей, роли, курсы, уроки, тесты, попытки, рекомендации и аналитические данные. Использование прямого SQL на всём протяжении проекта усложнило бы развитие и повторяемость изменений, тогда как связка SQLAlchemy + Alembic обеспечивает более управляемое развитие схемы.

1.5.4 Обоснование выбора технологий клиентских приложений

В административной части проекта рассматривались React + TypeScript, Vue и Angular. Для веб-панели требуются страницы авторизации, управления пользователями, курсами, уроками, тестами, назначениями и аналитикой. Такой интерфейс состоит из большого числа форм, таблиц и маршрутов, которые должны переиспользовать общие компоненты и работать с типизированными данными. По этой причине выбор был сделан

в пользу React как гибкой компонентной библиотеки, а TypeScript был добавлен для уменьшения количества ошибок при работе с API-моделями, ролями, курсами и результатами тестирования[39, 40].

Angular предоставляет более жёсткую архитектурную модель, однако для MVP это увеличивает объём шаблонного кода и порог входа. Vue также является сильным кандидатом, но в рассматриваемом проекте приоритет был отдан экосистеме React, которая хорошо сочетается с компонентной структурой административной панели, маршрутизацией и быстрым циклом сборки через Vite. Таким образом, выбор React + TypeScript оказался наиболее удобным именно для текущего состава интерфейсных сценариев.

Для мобильного клиента рассматривались Flutter, React Native и нативная Android-разработка. Так как в рамках ВКР необходимо было показать самостоятельный mobile-клиент слушателя, поддерживающий авторизацию, просмотр курсов, воспроизведение медиа, прохождение тестов и получение рекомендаций, требовалось решение с единым кодом приложения и предсказуемой сборкой интерфейса. Flutter отвечает этим требованиям и позволяет реализовать мобильный интерфейс как самостоятельную часть комплекса, не смешивая его с web-клиентом[28].

Нативная Android-разработка на Kotlin обеспечила бы высокий уровень платформенной интеграции, но заметно увеличила бы трудоёмкость проекта. React Native позволил бы использовать близкий по идеологии стек, однако не снимал бы задачи отдельной мобильной адаптации и не давал бы существенного выигрыша по сравнению с уже выбранным web-стеком. Поэтому для данной ВКР Flutter оказался более рациональным выбором.

1.5.5 Выбор СУБД и средств развёртывания

На уровне хранения данных в проекте используется двухконтурный подход. Для локальной разработки и части тестовых сценариев применяется

SQLite, так как она не требует запуска отдельного сервера, упрощает старт проекта и ускоряет проверку изменений. Для контейнеризированного развёртывания используется PostgreSQL, так как серверная СУБД лучше отражает рабочий режим многопользовательской системы, где одновременно функционируют серверная часть, web и база данных[41, 27].

Такое сочетание не является противоречием. Напротив, оно позволяет использовать простой режим разработки на ранних этапах и более реалистичный режим демонстрации на этапе развёртывания. Наличие в проекте конфигураций и для SQLite, и для PostgreSQL подтверждает, что выбранный стек ориентирован как на удобство разработки, так и на последующую демонстрацию результата.

Для локального развёртывания рассматривались ручной запуск сервисов, Docker Compose и более тяжёлые оркестрационные решения. Ручной запуск серверной части, web и базы данных неудобен для повторяемой демонстрации, так как требует последовательной настройки каждого компонента. Использование Kubernetes для учебного минимально жизнеспособного продукта (MVP) не даёт соразмерной выгоды и существенно усложняет инфраструктурную часть работы. По этой причине в проекте выбран Docker Compose, который позволяет в одном файле конфигурации описать сервисы базы данных, серверной части, web и при необходимости сервис начального заполнения данными[42].

Таким образом, текущий стек выбран как результат сопоставления требований проекта и трудоёмкости реализации. Он согласован со структурой каталогов исходного кода программного комплекса, обеспечивает поддержку веб- и мобильной платформ, удобен для развития предметной модели и позволяет воспроизводимо демонстрировать систему в рамках ВКР.

1.6 Выводы по главе

В первой главе рассмотрена предметная область корпоративного обучения, выявлены проблемы исходного состояния процессов и сформулировано назначение разрабатываемой системы. Определены функциональные и нефункциональные требования, приведена их проверка на основные свойства качества, а также сформирован сценарий тестирования требований по критическому пути пользователей. Выполнено сравнение аналогов и обоснован выбор собственного программного решения. На основании требований и анализа существующих платформ определён технологический стек проекта, соответствующий фактической реализации и задачам выпускной квалификационной работы.

2 ТЕХНИЧЕСКАЯ РЕАЛИЗАЦИЯ СИСТЕМЫ

2.1 Архитектура системы

Разрабатываемая система реализована как модульный монолит с REST API: серверная часть, веб-клиент и мобильный клиент составляют единый комплект исходных текстов (см. <https://github.com/lunahobi/coursum>), логически разделённый по каталогам. Такой подход упрощает развитие учебного проекта, позволяет явно показать взаимодействие подсистем и при этом сохраняет понятную декомпозицию по функциональным модулям[43, 18, 20].

Серверная часть выступает единственным источником бизнес-логики и данных. Она отвечает за аутентификацию, выбор текущей организации, проверку ролей, работу с курсами и уроками, управление тестами, выполнение адаптивного тестирования, формирование рекомендаций и подготовку аналитики. Веб-клиент ориентирован на роли администратора и преподавателя. Мобильный клиент ориентирован на роль слушателя и закрывает сценарии прохождения обучения.

На рисунке 3 представлена контейнерная схема системы. Диаграмма показывает состав программного комплекса на уровне контейнеров и основные каналы взаимодействия между веб- и мобильным клиентами, REST API серверной части, базой данных и хранилищем медиафайлов[44].

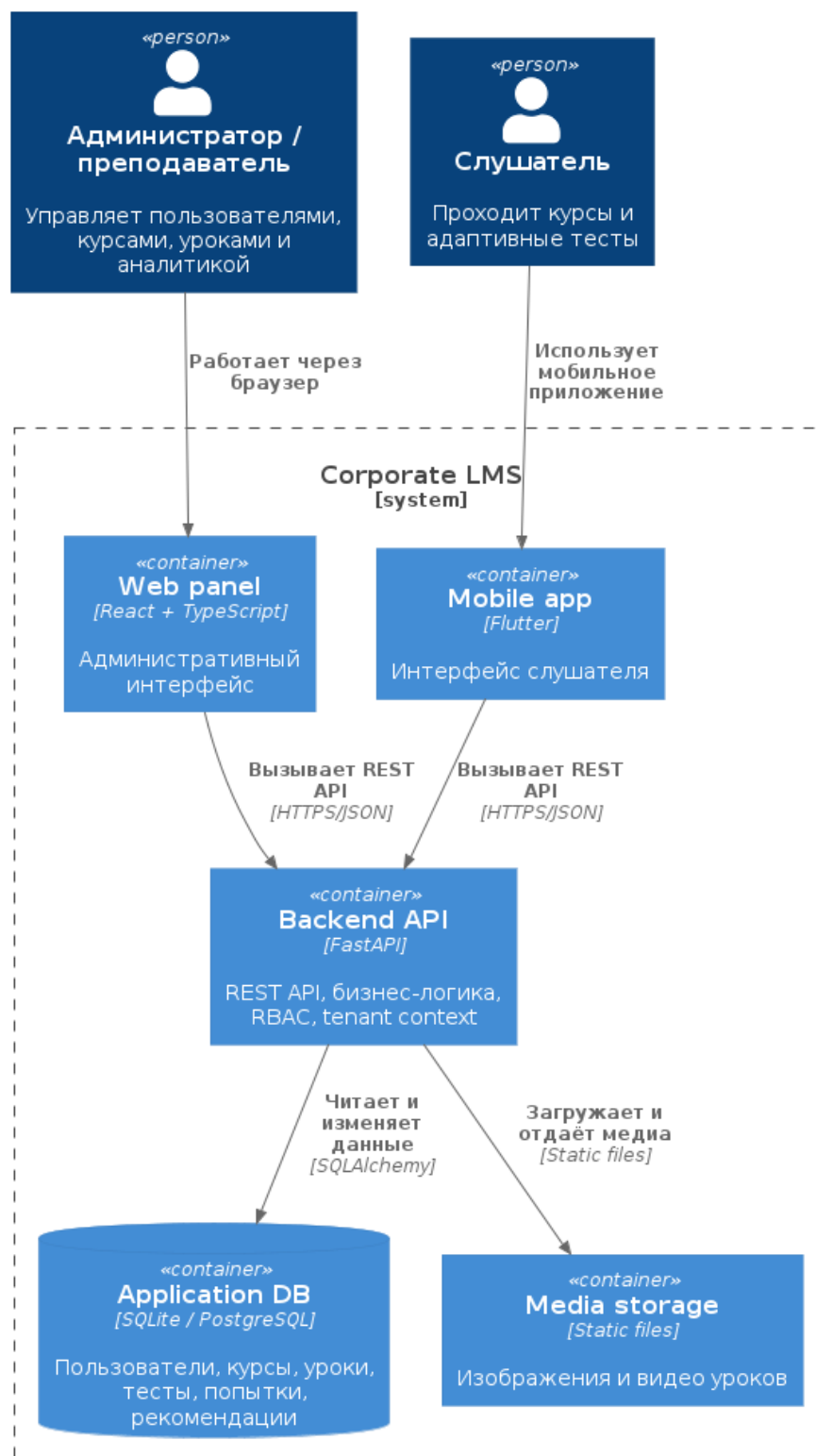


Рисунок 3 – Контейнерная схема программного комплекса

С точки зрения программной структуры серверная часть включает следующие подсистемы[26, 30, 29]:

- 1) auth — аутентификация, токены доступа и профиль пользователя;
- 2) tenants — работа с организациями и текущим контекстом организации-арендатора;

- 3) users — управление пользователями и их состоянием;
- 4) courses и lessons — управление курсами, уроками и прогрессом;
- 5) media — загрузка и выдача медиафайлов;
- 6) tests — жизненный цикл тестов и попыток;
- 7) adaptive — выбор следующего вопроса и расчёт рекомендаций;
- 8) recommendations — выдача списка рекомендаций слушателю;
- 9) analytics — агрегированная аналитика по системе.

На рисунке 4 приведена диаграмма компонентов серверной части. Она отражает декомпозицию серверного приложения на функциональные подсистемы и их связи с общими инфраструктурными механизмами.

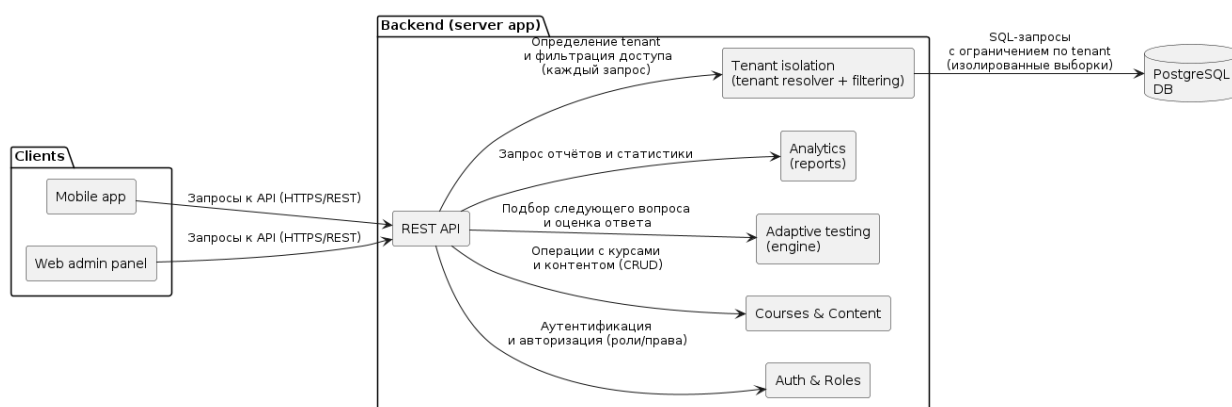


Рисунок 4 – Диаграмма компонентов системы

2.2 Сценарии использования

Во второй главе сценарии использования описываются как формальные последовательности действий, служащие основой для архитектурных и интерфейсных решений. Такое представление позволяет зафиксировать роли, предусловия, основной поток и результат выполнения для каждого ключевого сценария[6, 45].

На основании диаграммы прецедентов и критического пути пользователей в таблице 9 приведено табличное представление ключевых сценариев, используемое для согласования ролей, предусловий и результатов выполнения.

Таблица 9 – Табличное представление основных сценариев использования

Сценарий	Актор	Предусловие	Результат
Подготовка обучения и назначение курса	Преподаватель, администратор организации	Пользователь аутентифицирован и работает в контексте своей организации	Курс создан, наполнен контентом и назначен слушателю.
Прохождение курса и адаптивного теста	Слушатель	Слушатель включён в организацию и имеет назначенный курс	Сформирован результат попытки и набор рекомендаций по слабым темам.
Анализ результатов обучения	Преподаватель, администратор организации	В системе накоплены данные по назначениям и попыткам	Получены агрегированные показатели и детализированные результаты по слушателям.

2.2.1 Сценарий 1: подготовка обучения и назначение курса

Акторы: администратор организации, преподаватель.

Предусловия: пользователь аутентифицирован, организация выбрана, роль пользователя позволяет управлять контентом и назначениями.

Основной поток:

- 1) преподаватель создаёт курс;
- 2) преподаватель добавляет уроки и при необходимости медиа-контент;
- 3) преподаватель создаёт тест и вопросы;
- 4) администратор организации выбирает слушателя;
- 5) администратор назначает курс слушателю;
- 6) система фиксирует назначение и делает курс доступным в мобильном клиенте слушателя.

Постусловия: слушатель получает назначенный курс и может приступить к изучению материала.

2.2.2 Сценарий 2: прохождение курса и адаптивного теста

Актор: слушатель.

Предусловия: слушатель аутентифицирован, имеет активное участие в организации, курс назначен, тест связан с курсом.

Основной поток:

- 1) слушатель открывает список своих курсов;
- 2) слушатель переходит в курс и изучает урок;
- 3) слушатель запускает тест;
- 4) система создаёт попытку с базовой сложностью;
- 5) система выдаёт первый вопрос;
- 6) слушатель отправляет ответ;
- 7) система оценивает ответ и изменяет текущую сложность;
- 8) система выдаёт следующий вопрос с учётом новой сложности;
- 9) после достижения лимита вопросов система завершает попытку;
- 10) система сохраняет результат, слабые темы и формирует рекомендации.

Постусловия: результат попытки, история сложности и рекомендации доступны слушателю и административным ролям.

2.2.3 Сценарий 3: анализ результатов обучения

Актёры: преподаватель, администратор организации.

Основной поток:

- 1) пользователь открывает раздел аналитики;
- 2) система отображает агрегированные показатели по курсам, тестам и назначенным обучениям;
- 3) пользователь открывает проблемные темы или индивидуальный отчёт по слушателю;
- 4) система отображает результаты попыток, количество правильных ответов и рекомендации.

2.3 Модель данных

Модель данных построена вокруг сущностей, необходимых для поддержки мультиарендного обучения (несколько организаций в одном экземпляре)[19, 46, 47, 48]. На концептуальном уровне она отражает объекты предметной области и их семантические связи, а на логическом уровне фиксирует состав реляционных таблиц, ключей и ограничений целостности. Центральной сущностью является организация (tenant), относительно которой изолируются пользователи, курсы, уроки, тесты, попытки и рекомендации.

2.3.1 Инфологическая модель базы данных

Инфологическая модель описывает систему без привязки к конкретной СУБД и показывает укрупнённые сущности предметной области: организацию, пользователя, курс, урок, тему, тест, вопрос, попытку и результат. Такая степень абстракции позволяет наглядно показать логику движения данных от подготовки учебного контента до прохождения адаптивного теста и получения результата.

На рисунке 5 приведена инфологическая модель базы данных. На этом уровне намеренно опущены служебные и ассоциативные сущности, связанные с ролями, назначениями, рекомендациями и аудитом, так как они раскрываются в даталогической модели.

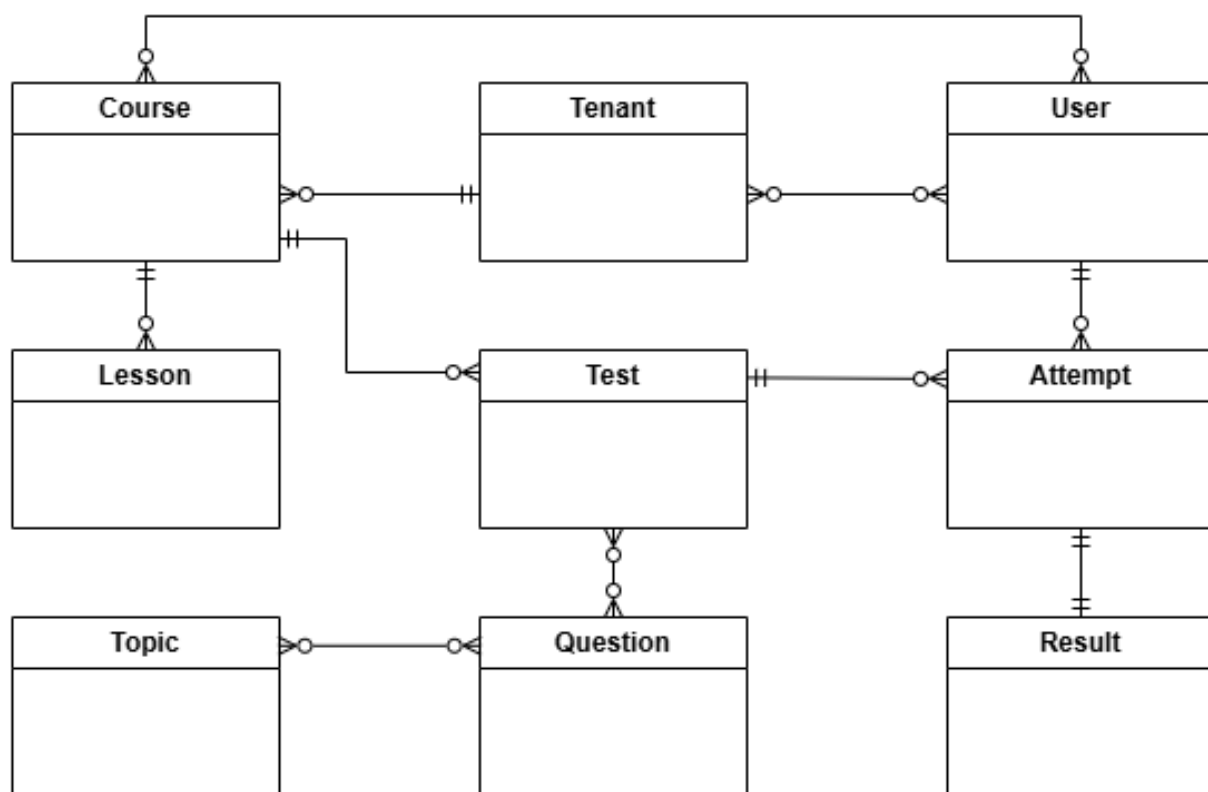


Рисунок 5 – Инфологическая модель базы данных

2.3.2 Даталогическая модель базы данных

На даталогическом уровне инфологическая модель преобразуется в реляционную схему, реализованную средствами SQLAlchemy. При локальном запуске приложение может использовать SQLite, а при контейнеризированном развёртывании — PostgreSQL; при этом набор таблиц, первичных и внешних ключей остаётся одинаковым. Для tenant-зависимых сущностей в схему включено поле `tenant_id`, а для критичных связей заданы ограничения уникальности, исключающие дублирование членств, назначений и записей о прогрессе.

Для пояснительной записки даталогическая модель представлена двумя основными фрагментами, отражающими базовые контуры проектируемой системы: организационный контур и контур управления учебным контентом. Такой способ позволяет показать структуру таблиц и связей без перегрузки рисунков второстепенными служебными сущностями.

На рисунке 6 показана даталогическая модель организационного контура. Она описывает, каким образом в системе представлены организации, пользователи, роли, членства и учебные группы. Данный фрагмент показывает реализацию мультиарендного подхода: один пользователь может состоять в нескольких организациях, а его права определяются через таблицу членства и связанную с ней роль. Дополнительно здесь отражено объединение пользователей в группы, используемые при массовом назначении обучения.

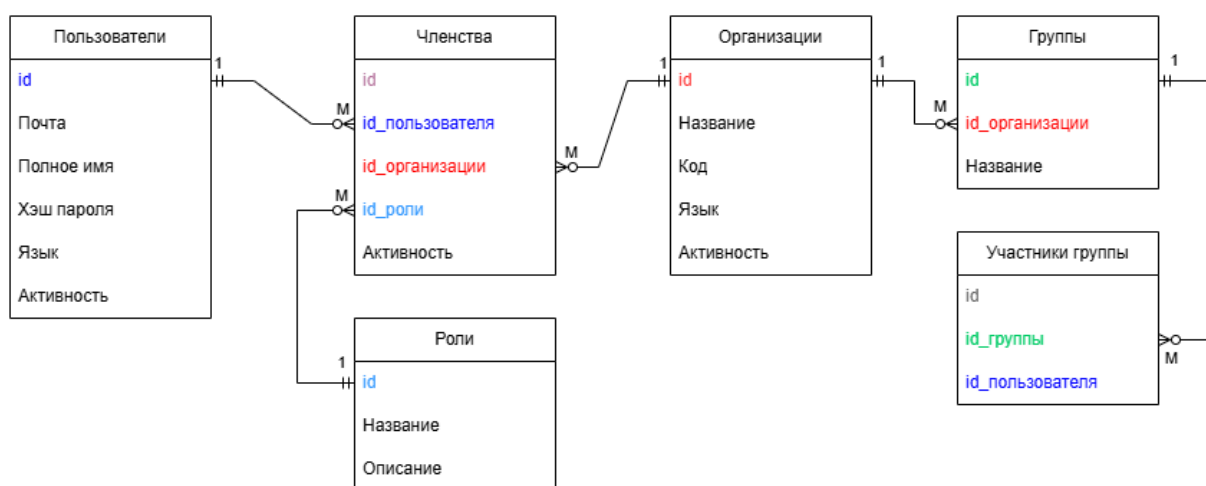


Рисунок 6 – Даталогическая модель организационного контура

На рисунке 7 представлена даталогическая модель контура курсов. В неё включены таблицы, описывающие учебный контент, структуру курса, назначения, зачисления и фиксацию прогресса. Схема показывает, что курс принадлежит организации, состоит из уроков и тем, может назначаться как отдельному пользователю, так и группе, а факт прохождения отражается в таблицах зачислений и прогресса по урокам. Такой контур обеспечивает хранение как структуры учебных материалов, так и состояния их освоения слушателями.

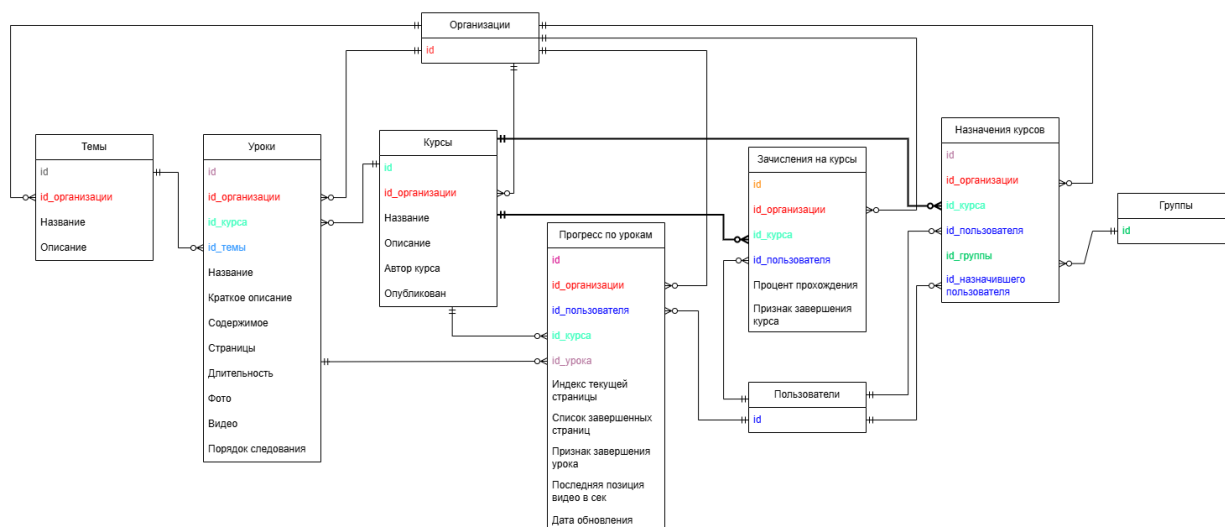


Рисунок 7 – Даталогическая модель контура курсов

Таким образом, в текущей версии пояснительной записки даталогическая модель раскрыта через два ключевых фрагмента: организационный и учебный. Первый описывает разграничение доступа и принадлежность пользователей к организациям и группам, а второй — структуру курсов, механизм их назначения и фиксацию прогресса обучения. Контуры тестирования, рекомендаций и служебных таблиц при необходимости могут быть представлены отдельно без перегрузки основной части работы.

2.4 Проектирование интерфейсов

Проектирование интерфейсов выполнялось отдельно для административного и пользовательского контуров с учётом принципов проектирования взаимодействия[49, 50, 51].

2.4.1 Веб-интерфейс администратора и преподавателя

Веб-клиент предназначен для работы с учебным контентом и аналитикой. Его основными разделами являются:

- 1) вход в систему и выбор организации;
- 2) управление пользователями;

- 3) управление курсами;
- 4) управление уроками и медиатекой;
- 5) управление тестами;
- 6) назначение курсов;
- 7) просмотр аналитики;
- 8) настройки профиля и языка интерфейса.

Структура веб-интерфейса ориентирована на последовательное выполнение административных операций: сначала создаётся контент, затем выполняются назначения, после чего доступны данные аналитики.

2.4.2 Мобильный интерфейс слушателя

Мобильный клиент ориентирован на компактный пользовательский сценарий:

- 1) вход пользователя;
- 2) просмотр списка назначенных курсов;
- 3) открытие курса и урока;
- 4) запуск теста;
- 5) просмотр результата, рекомендаций и истории попыток.

На рисунках 8–9 показаны примеры экранов мобильного клиента.

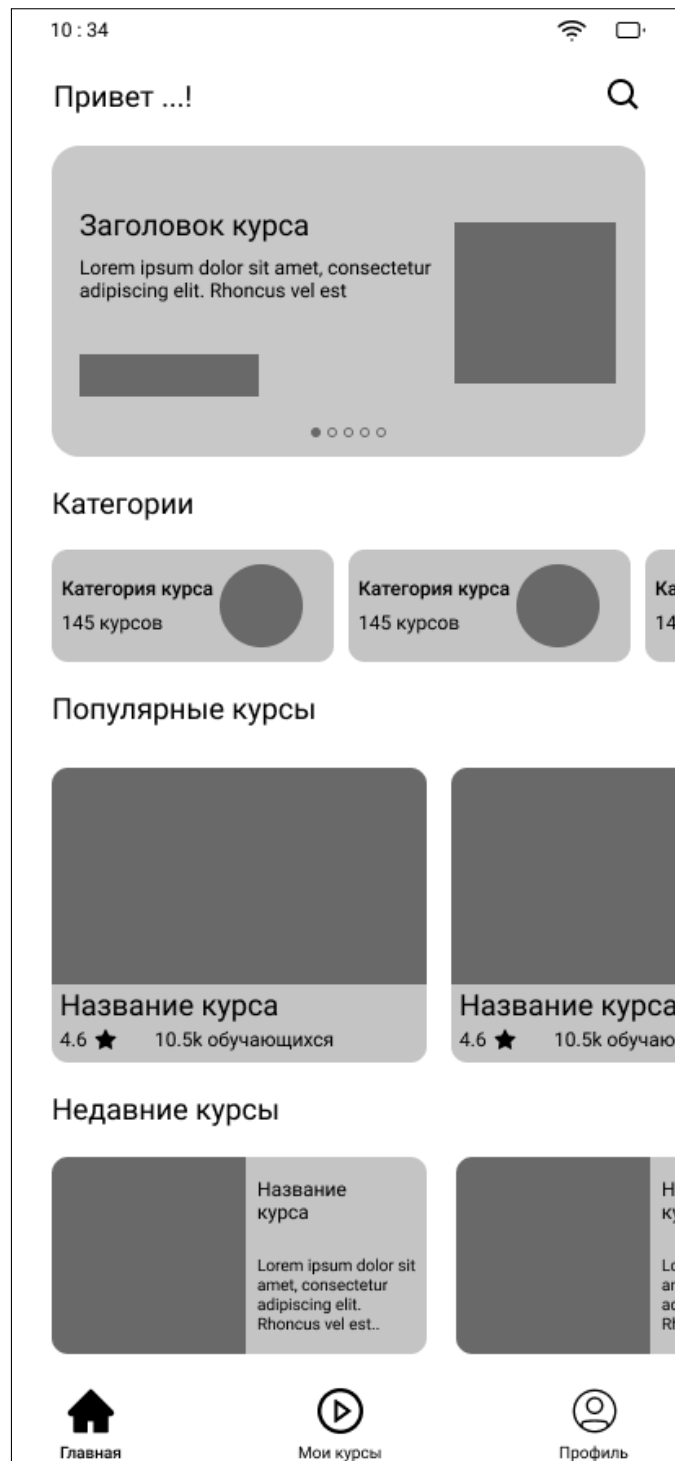


Рисунок 8 – Главный экран мобильного клиента

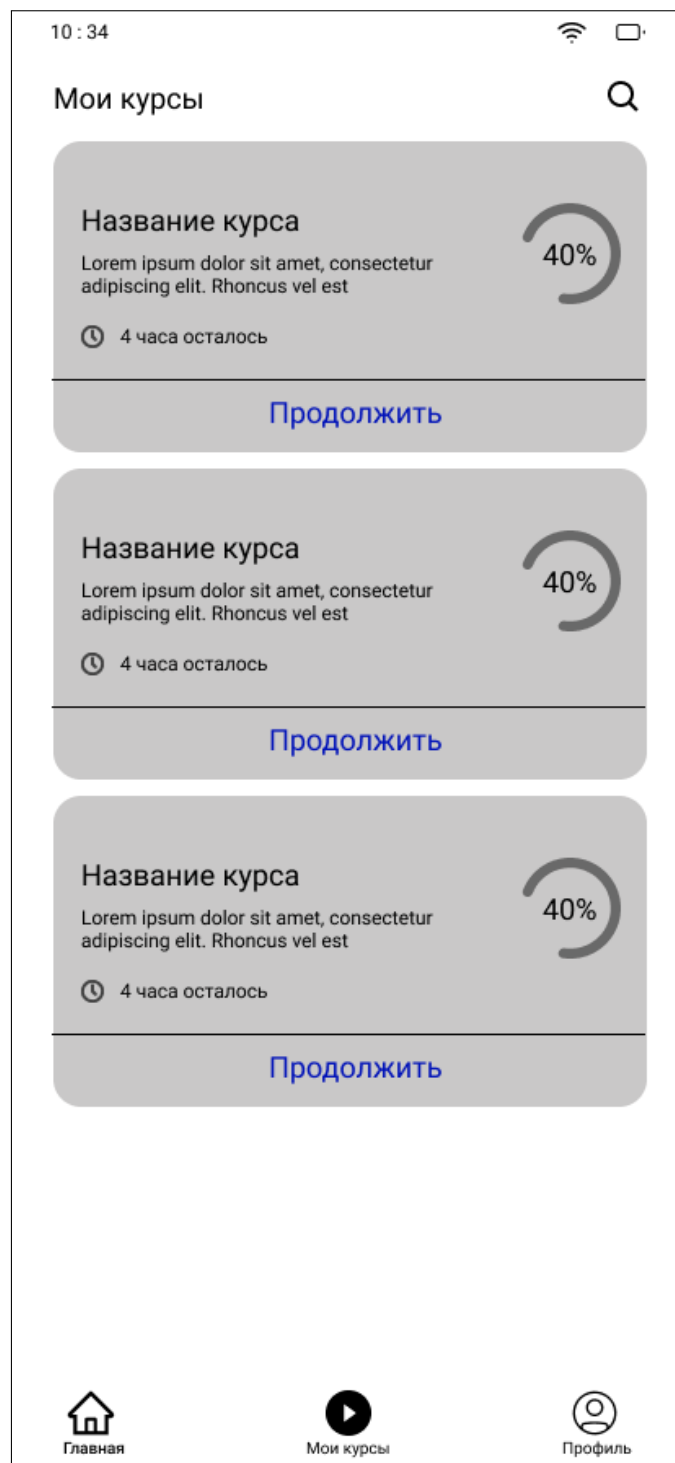


Рисунок 9 – Экран списка назначенных курсов

2.5 Реализация web- и mobile-клиентов

Клиентская часть системы реализована как два самостоятельных приложения, использующих единый REST API серверной части, но ориентированных на разные роли и сценарии работы. Веб-клиент предназначен для администратора и преподавателя, а мобильный клиент — для слушателя.

Такое разделение позволяет не перегружать интерфейсы лишними функциями и адаптировать взаимодействие с системой под реальные задачи пользователей[39, 28].

На рисунке 10 представлена структурная схема клиентской части системы. Она показывает внутреннее устройство web- и mobile-приложений: пользовательский интерфейс, прикладные экраны, слой взаимодействия с API серверной части и локальное хранение пользовательского состояния. Общим для обоих клиентов является единый контракт обмена данными, а различие определяется набором экранов и способом управления состоянием.

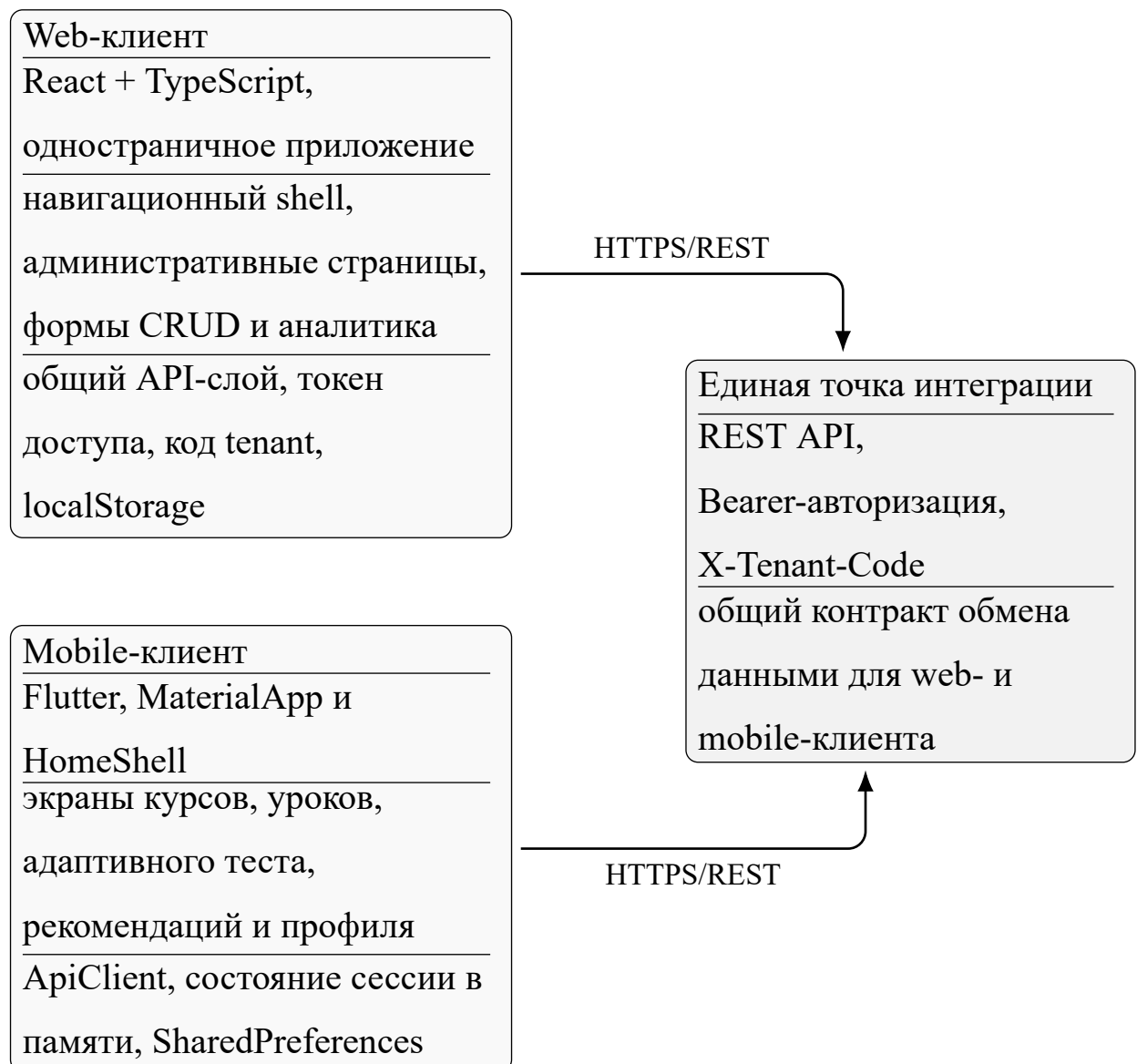


Рисунок 10 – Структурная схема клиентской части системы

2.5.1 Реализация web-клиента

Веб-клиент реализован как одностраничное приложение на React и TypeScript. После аутентификации пользователь попадает в общий каркас приложения с боковой навигацией, из которого доступны основные разделы: панель показателей, переключение организации, пользователи, курсы, уроки, тесты, назначения, аналитика и настройки. Маршрутизация организована на стороне клиента, что позволяет быстро переходить между разделами без повторной загрузки страницы.

Работа с серверной частью вынесена в отдельный API-слой. Для всех запросов централизованно задаются базовый адрес сервера, заголовок авторизации Bearer и заголовок X-Tenant-Code, определяющий текущую организацию. Такой подход уменьшает дублирование кода и обеспечивает единое поведение для операций чтения, создания, редактирования и загрузки медиафайлов. Для типовых запросов используются отдельные функции отправки GET, POST, PATCH и multipart/form-data.

Сессионное состояние web-клиента включает токен доступа и код текущей организации. Эти данные передаются между страницами через состояние верхнего уровня приложения. Дополнительно в интерфейсе реализовано переключение языка, причём выбранный язык сохраняется в localStorage браузера и применяется при следующем открытии приложения.

Функциональные страницы web-клиента построены по единому принципу: загрузка данных через общий механизм запросов, отображение списка сущностей, форма создания или редактирования и визуальное подтверждение результата операции. Так реализованы управление пользователями, курсами, уроками, тестами и назначениями. В разделе уроков предусмотрено редактирование структуры страниц урока и загрузка медиафайлов, а в разделе аналитики — загрузка агрегированных показателей, проблемных тем и данных по конкретному слушателю. В результате веб-клиент закрывает весь ад-

министративный контур системы и напрямую поддерживает сценарии, описанные в первой главе.

2.5.2 Реализация мобильного клиента

Мобильный клиент реализован на Flutter как отдельное приложение слушателя. Корневой экран приложения построен на базе MaterialApp, а после входа пользователь переходит в основной HomeShell, содержащий нижнюю навигацию между тремя разделами: курсы, рекомендации и профиль. Для переключения между разделами используется IndexedStack, что позволяет сохранять состояние экранов при переходах.

Взаимодействие с сервером инкапсулировано в классе ApiClient. Он отвечает за формирование HTTP-запросов, добавление заголовков авторизации и заголовка организации (X-Tenant-Code), обработку ошибок и преобразование JSON-ответов в модели приложения. Через этот клиент выполняются аутентификация, загрузка курсов, получение структуры курса, получение рекомендаций, работа с попытками тестирования и загрузка данных для разбора завершённых попыток.

Состояние мобильного приложения разделено на два уровня. Языковые настройки сохраняются в SharedPreferences, благодаря чему выбор русского или английского языка восстанавливается после перезапуска приложения. Сессионные данные текущего пользователя передаются между экранами в памяти приложения, что достаточно для учебного минимально жизнеспособного продукта (MVP) и упрощает жизненный цикл клиента.

Экран списка курсов загружает назначенные пользователю курсы и открывает детальный экран курса. На экране курса выполняется параллельная загрузка структуры курса и связанных с ним тестов, после чего слушателю становятся доступны три действия: продолжение обучения, запуск адаптивного теста и просмотр истории попыток. Для прохождения урока использует-

ся отдельный экран-плеер, который получает данные урока с сервера и сохраняет прогресс пользователя, включая текущую страницу, набор завершённых страниц и позицию воспроизведения видео.

Отдельный экран `TestFlowScreen` реализует клиентскую часть адаптивного тестирования. Он инициирует попытку, получает очередной вопрос, отправляет выбранный ответ с учётом времени реакции и завершает попытку после исчерпания набора вопросов. После завершения теста слушатель видит результат, слабые темы и рекомендации. Дополнительно в мобильном клиенте реализованы экран рекомендаций и экран истории попыток с возможностью открыть детальный разбор завершённого теста. Таким образом, мобильный клиент не ограничивается просмотром контента, а поддерживает полный пользовательский цикл слушателя: от изучения курса до анализа собственных результатов.

2.6 Реализация серверной логики и алгоритмов

2.6.1 Реализация API-подсистем

Серверная часть реализует REST API, сгруппированное по назначению маршрутов. Такое разбиение определяет формат взаимодействия клиентов с серверной частью и набор доступных операций.

- 1) `/auth` — регистрация, вход, обновление токенов, получение профиля;
- 2) `/tenants` — список доступных организаций, текущая организация, выбор организации;
- 3) `/users` — просмотр, создание и изменение пользователей;
- 4) `/courses` и `/lessons` — работа с курсами, уроками, прогрессом и назначениями;
- 5) `/media` — библиотека и загрузка медиа;
- 6) `/tests` и `/attempts` — тесты, вопросы, попытки, история и разбор;

- 7) /recommendations — рекомендации слушателя;
- 8) /analytics — сводная и детальная аналитика.

Такое разбиение соответствует структуре кода серверной части и обеспечивает независимое развитие функциональных подсистем при сохранении единой точки входа.

На рисунке 11 приведена структурная схема серверной части системы. В отличие от общей диаграммы компонентов, эта схема акцентирует именно уровни обработки запроса: маршрутный слой, контур tenant-контекста и разграничения доступа, сервисный слой бизнес-логики и слой хранения данных.

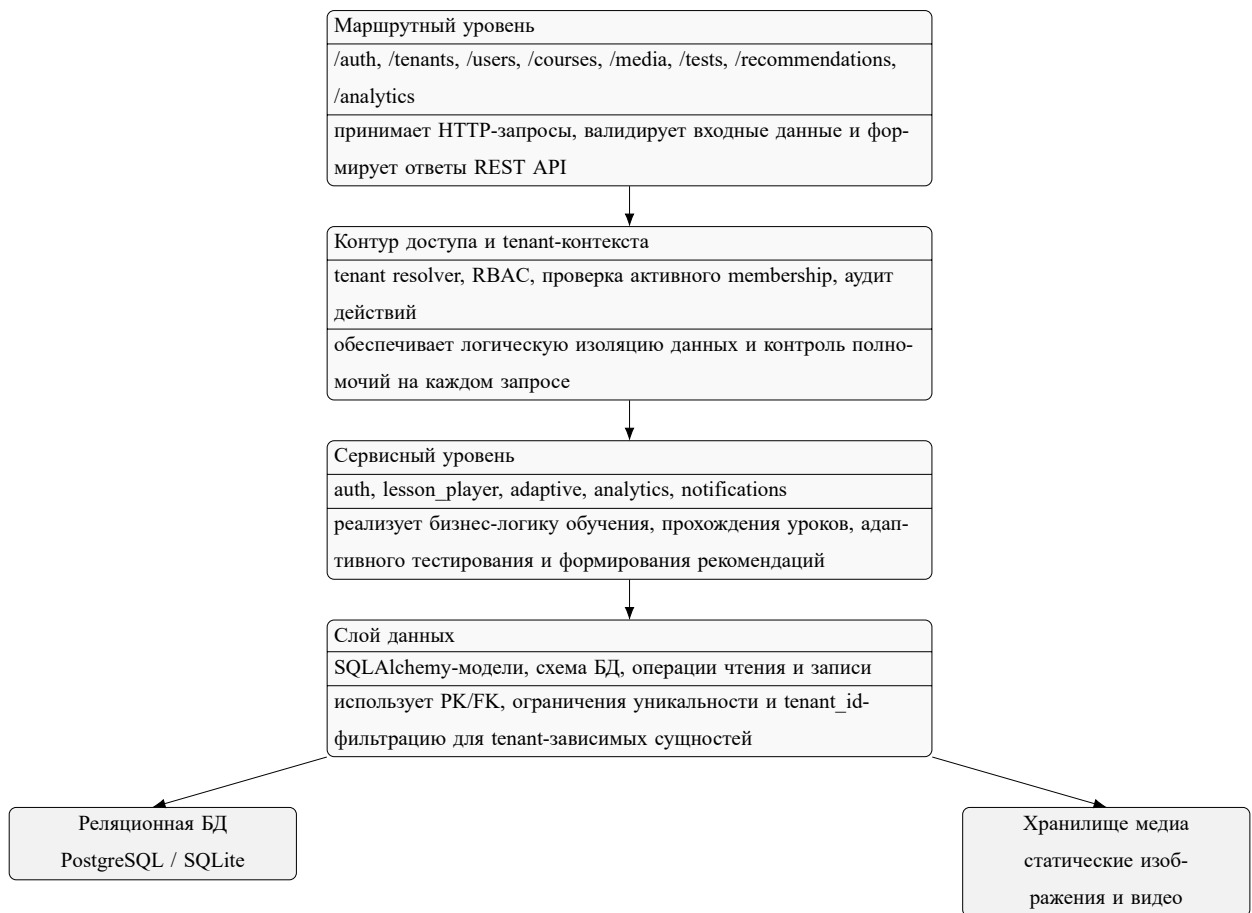


Рисунок 11 – Структурная схема серверной части системы

2.6.2 Алгоритм адаптивного тестирования

В проекте реализован дискретный алгоритм адаптивного тестирования, работающий в диапазоне сложностей от 1 до 5[25, 23, 24]. Алгоритм основан на следующих правилах:

- 1) при старте попытки текущая сложность равна базовой сложности теста;
- 2) после правильного ответа сложность повышается на один уровень;
- 3) после неправильного ответа сложность понижается на один уровень;
- 4) следующее задание выбирается среди ещё не заданных вопросов, наиболее близких к текущей сложности;
- 5) по ошибочным и медленным ответам накапливаются слабые темы;
- 6) после завершения попытки формируются рекомендации по повторению материала.

Структурные фрагменты модулей `adaptive`, интерфейса тестов веб-панели и клиентского экрана `TestFlowScreen` см. приложение 3.6.

На рисунке 12 показана последовательность взаимодействия клиента и сервера при прохождении адаптивного теста. Диаграмма фиксирует обмен запросами между мобильным приложением, REST API серверной части и модулем адаптивной логики.

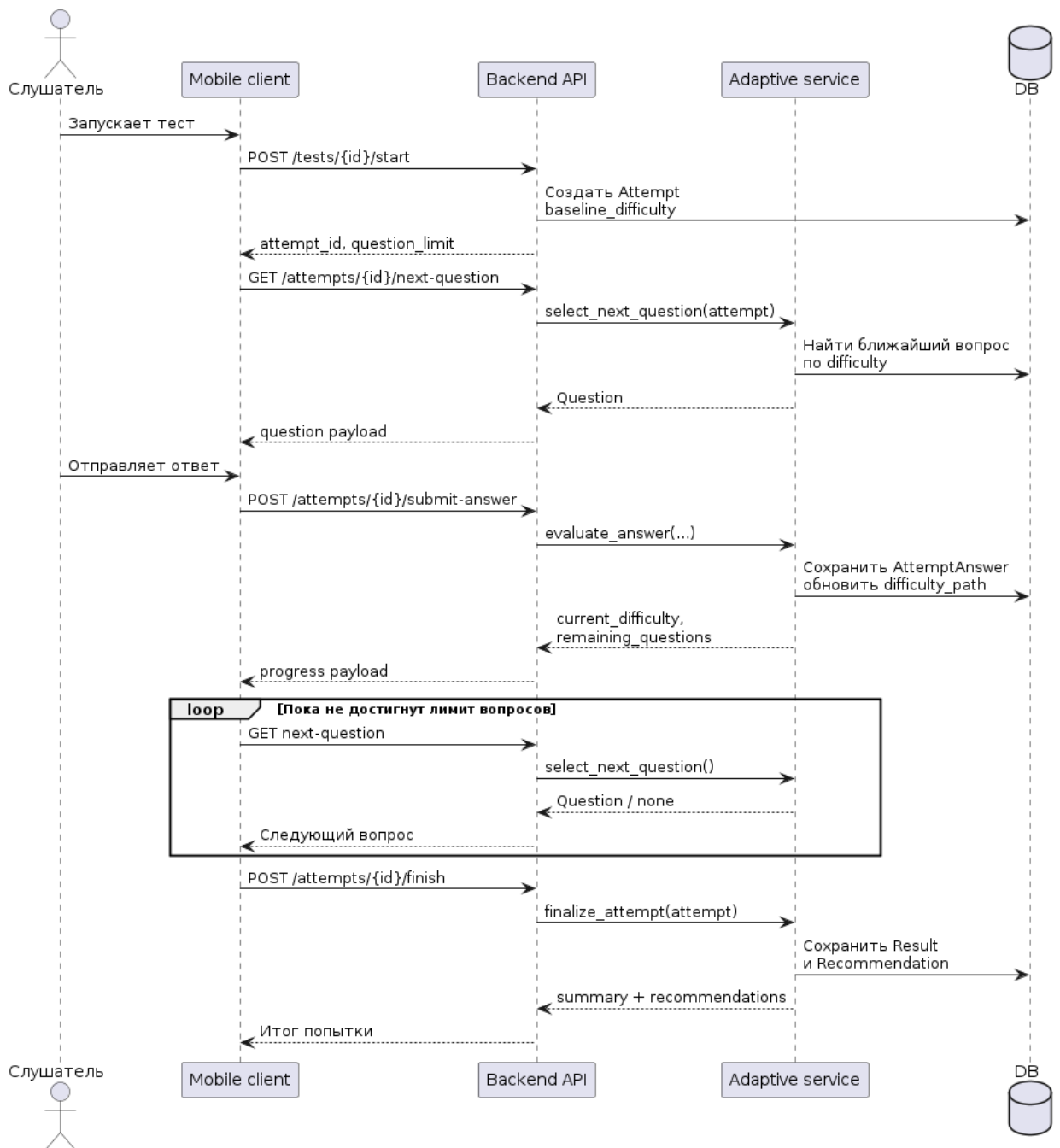


Рисунок 12 – Последовательность прохождения адаптивного теста

Для иллюстрации логики изменения сложности на рисунке 13 приведена схема алгоритма. Она отражает старт с базовой сложности, выбор ближайшего незаданного вопроса, изменение уровня сложности на шаг '+1/-1' и формирование рекомендаций после завершения попытки.

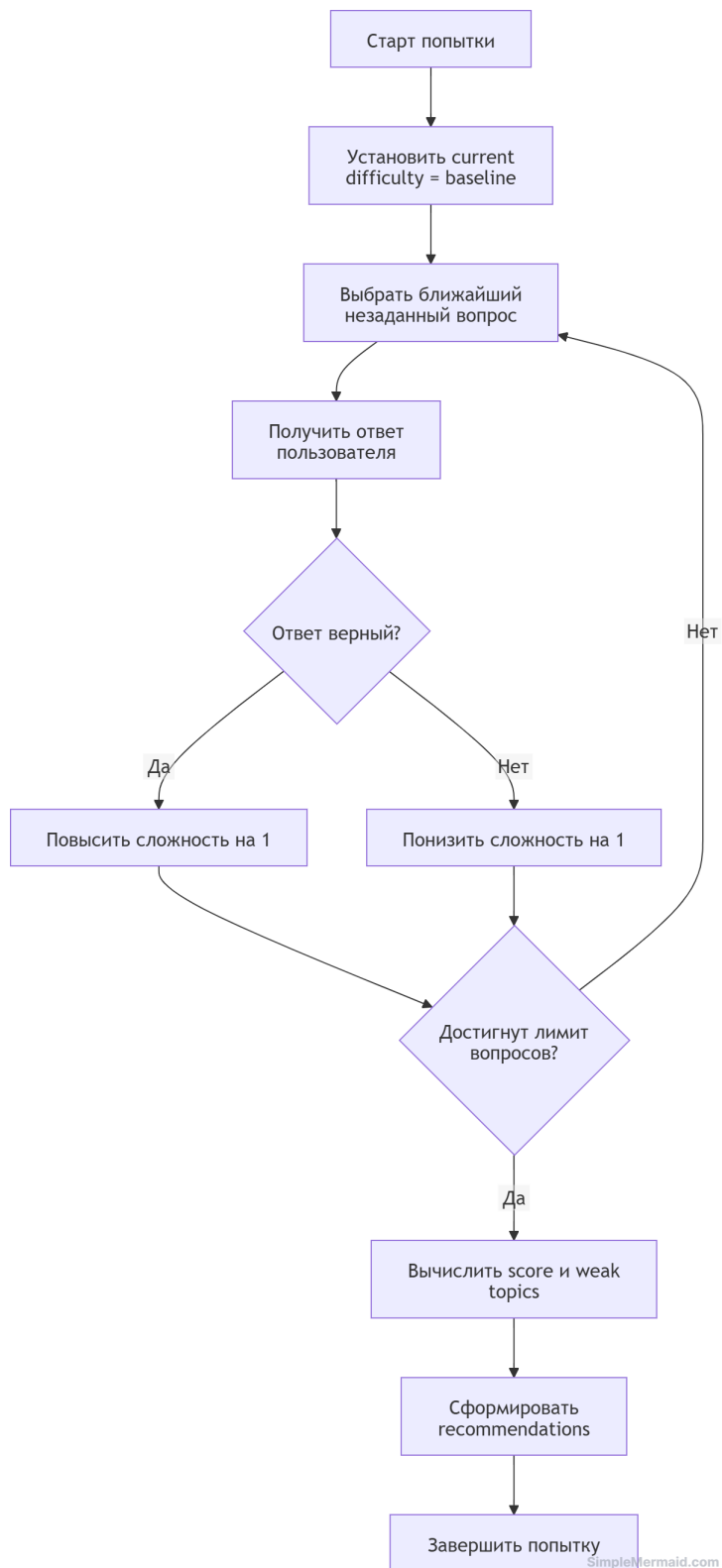


Рисунок 13 – Схема изменения сложности в адаптивном тестировании

2.6.3 Изоляция данных арендаторов

Мультиарендная изоляция данных реализована на трёх уровнях[5, 52]:

- 1) на уровне запроса определяется код текущей организации;

2) на уровне авторизации проверяется, что пользователь имеет активное участие в этой организации;

3) на уровне доступа к данным выполняется фильтрация сущностей, зависящих от организации-арендатора, по полю `tenant_id`.

Таким образом, логическая изоляция не требует выделения отдельной базы данных на каждый объект аренды и при этом обеспечивает корректное разделение данных в рамках минимально жизнеспособного продукта (MVP).

2.7 Выводы по главе

Во второй главе описана техническая реализация корпоративной LMS. Рассмотрены архитектура системы в виде модульного монолита, формальные сценарии использования, инфологическая и даталогическая модели базы данных (включая обоснование нормализации и связей между контурами организации, учебного контента и тестирования), проектирование интерфейсов веб-клиента и мобильного клиента, а также структурные схемы клиентской и серверной частей. Раскрыты состав REST API, алгоритмы адаптивного тестирования, формирования рекомендаций и логической изоляции данных арендаторов на уровне запроса, авторизации и доступа к данным. Процедуры контейнеризированного развёртывания, опытной эксплуатации и инструкции для пользователей и администраторов вынесены в третью главу и приложения пояснительной записки. Представленные решения соответствуют фактической реализации проекта и обеспечивают основу для демонстрации работоспособности системы на защите [6, 18, 44].

3 ВНЕДРЕНИЕ И ЭКСПЛУАТАЦИЯ ПРОГРАММНОГО КОМПЛЕКСА

3.1 Анализ внедрения и оценка результатов реализации

Под внедрением понимается опытное развёртывание программного комплекса в контролируемой среде и проверка соответствия фактической реализации функциональным и нефункциональным требованиям, сформулированным в главе 1 по матрице трассировки критического пути пользователей[1, 15, 12].

Последовательность опытного развёртывания.

1) Подготовить среду: Docker Engine и Docker Compose в соответствии с требованиями к вычислительным ресурсам; установить среду Flutter для сборки мобильного приложения слушателя.

2) Клонировать исходный код из репозитория <https://github.com/lunahobi/coursum> и настроить параметры по образцу `.env.example` (`LMS_SECRET_KEY`, `LMS_DATABASE_URL` при необходимости, `LMS_ENVIRONMENT`, `LMS_CORS_ORIGINS`, параметры доступа к СУБД).

3) Запустить контейнеры командой `docker compose up -d --build`: сервис `db` использует образ `postgres:16-alpine` и публикацию порта `127.0.0.1:5433→5432`, серверная часть слушает `127.0.0.1:8000`, контейнер веб-панели — `127.0.0.1:8081`.

4) После перехода СУБД в состояние готовности `entrypoint` образа серверной части выполняет миграции Alembic (`alembic upgrade head`) и запускает `uvicorn app.main:app`.

5) Загрузить демонстрационные данные: `docker compose run --rm --profile tools seed` (эквивалент локально без Docker: в каталоге `backend` выполняется `python -m scripts.seed_demo`).

б) Проверить доступность наблюдаемости: запросы GET /health и GET /health/db; для REST-контура — префикс /api/v1.

Мобильный клиент выполняется вне контейнерного профиля: из каталога mobile выполняется flutter pub get, затем flutter run с при необходимости переопределением базового URL API. Для эмулятора Android обычно задают --dart-define=API_BASE=http://10.0.2.2:8000/api/v1; на физическом устройстве вместо 10.0.2.2 подставляют IP-адрес узла сети, на котором слушает серверная часть[28, 42, 26].

Опытное развёртывание в сети Интернет. Помимо локального стенда выполнено развёртывание на арендованном виртуальном выделенном сервере (VPS): зарегистрировано доменное имя coursum.online, для публичного доступа к REST API и выдачи загруженных медиафайлов (маршруты /media и связанные ресурсы серверной части) используется поддомен https://api.coursum.online с базовым префиксом API /api/v1. Сборка образов и запуск в этом контуре выполнялись через Docker Compose; взаимодействие веб-панели с серверной частью и политика CORS настраиваются на источники с домена coursum.online. Порядок развёртывания и пример публичного контура описаны в приложении Г[1, 11].

Интерпретация прогноза и результата. В таблице 10 сопоставлены целевые требования (источник прогноза — главы 1–2 и техническое задание приложения А): способ проверки задаёт наблюдаемый критерий, результат фиксирует вывод без выдуманных численных данных.

Таблица 10 – Сопоставление требований, способа проверки и результата при опытном внедрении

Требование	Способ проверки	Результат (опытное внедрение)
Клиент-серверный контур REST /api/v1, запись в PostgreSQL опытном стенде	смок-доступ к маршрутам API и корректные ответы при выполнении сценариев приложения Б	пройдено на локальном стенде после миграций и сидирования
Роли пользователей (learner, teacher, org_admin, system_admin)	попытки вызова маршрутов из-под каждой демонстрационной роли	соответствие матрице доступа приложения Б
Логическая мультиарендная модель данных (контекст организации без смещения tenant_id)	тест-кейсы на изоляцию; регистрация двух пользователей с одинаковым адресом электронной почты и различными членствами	изоляция по запросам с контекстом организации не нарушалась
Рекомендации после попытки (ошибочные ответы и медленные ответы)	финиш попытки тестирования слушателя, просмотр рекомендаций	наблюдалось после обработки /attempts/{id}/finish
Конфигурируемая политика CORS и поддержка локального режима заголовка организации (X-Tenant-Code)	инспекция LMS_CORS_ORIGINS; вызовы с заголовком в dev-среде	соответствует настройке app.main:app; разграничение источников в продакшене — за обратным прокси
Публичный контур HTTPS, домен coursum.online, API и медиа на api.coursum.online	проверка сертификата, заголовков CORS, GET к /health и загрузка медиа по выданному URL	подтверждено на VPS при развёртывании через Docker Compose

По качественным результатам установлено, что цели ВКР по созданию воспроизводимого стенда, прохождения пользовательского критического пути и демонстрации адаптивного поведения тестирования достигаются без блокирующих отклонений от зафиксированных требований. Количественные метрики прогона и времени отклика см. ниже как отложенное заполнение.

Примечание про иллюстративную схему стенда. По исходнику диаграммы diagrams/deployment.puml экспортируемый рисунок включают в главу после верстки.

3.2 Тестирование программного комплекса

План испытаний соответствует ГОСТ 19.301–79 как основе для приложения Б[16, 45]. В ходе проекта выполняются: функциональное тестирование по пользовательским сценариям критического пути; интеграционное тестирование взаимодействий клиентская часть (веб-панель) ↔ серверная часть, серверная часть ↔ PostgreSQL; смок-тестирующие проверки после развёртывания; проверки изоляции организаций-арендаторов для одновременно существующих контекстов.

В таблице 11 приведены ключевые тест-кейсы для текста главы; полное покрытие с не менее чем двадцатью наблюдаемыми процедурами зафиксировано в приложении Б[7, 22].

Таблица 11 – Ключевые тест-кейсы критического пути программного комплекса

ID	Кратко	Шаги	Ожидаемое	Факт /
ТС-K01	Вход преподавателя или администратора организации в веб-панель	авторизация, выбор активной организации, проверка маршрутизации приложения (X-Tenant-Code или прокси домена); см. прилож. В	доступ к разделам администрирования в рамках выбранной организации	пройден
ТС-K02	Создание курса, урока, теста и вопросов	использовать разделы курсов, уроков, тестов веб-панели; сохранять JSON-тела запросов; см. прилож. Б, ТС-блок создания контента	сущности отображаются в списках и доступны назначениям и попытке	пройден
ТС-K03	Назначение курса пользователю или группе	применить сценарий назначения из веб-панели; см. прилож. В	назначение отражено в данных слушателя	пройден
ТС-K04	Авторизация слушателя в мобильном приложении	учётные данные см. прилож. В; используется демонстрационный аккаунт слушателя	успешное получение пары JWT (access и refresh по ответам API) и загрузка курсов	пройден
ТС-K05	Прохождение урока с сохранением прогресса	открыть урок из назначений, сохранять страницу; проверять маршрут прогресса	прогресс отражён при повторном входе в урок	пройден
ТС-K06	Старт и прохождение адаптивного теста	/tests/{id}/start, получение следующего вопроса (/attempts/{id}/next-question), отправка ответа (/attempts/{id}/submit-answer); см. прилож. Б	наблюдаемое изменение current_difficulty на шаг ± 1 в пределах 1..5	пройден
ТС-K07	Завершение попытки, рекомендации слушателя	/attempts/{id}/finish, затем /recommendations/me; см. листинги прилож. Д	ответ содержит итоговый балльный показатель и перечень рекомендаций после ошибок или медленных ответов	пройден
ТС-K08	Аналитические сводки в веб-панели	открыть модуль сводной аналитики за роль преподавателя или администратора организации	агрегаты по курсам и слушателям отображаются без ошибок HTTP	пройден
ТС-K09	Изоляция арендаторов	создать или использовать две организации с одинаковыми локальными адресами участников отдельными членствами; запрос данных чужого tenant_id	ответы с кодами ошибок доступа, отсутствие «пересечения» выборок	пройден (разработка регрессии)
ТС-K10	Смок-после деплоя	/health, /health/db, вход в веб-панель после docker compose up -d; см. прилож. Г	готовность стенда к демонстрации без критических ошибок в журналах	пройден (локально)

Метрики опытной эксплуатации. Суммарное число прогонов unit- и сервисных тестов, число регрессий после изменений регламентируется журналами CI в репозитории на GitHub[53, 9].

Разъяснение по колонке «Фактический результат». По таблице 11 она указывает статус наблюдавшегося исхода демонстрационного или регресси-

онного прогона; при защите подставляются ссылки на отчёт (СІ, приложение Б)[7, 8].

3.3 Аспекты информационной безопасности

3.3.1 Модель угроз

Для корпоративной системы управления обучением в мультиарендном развёртывании доминируют угрозы нарушения конфиденциальности результатов обучения и профилей, компрометации токена доступа, повышения привилегий в обход ограничения ролевой модели (RBAC), нарушение изоляции данных между организациями-арендаторами, атаки внедрения на интерфейсах API, подбор учётных данных (перебор паролей) против конечной точки аутентификации[14, 13, 47].

На рисунке 14 приведена упрощённая схема соответствия классов угроз взаимно дополняющимся наблюдением по периметру системы управления обучением.

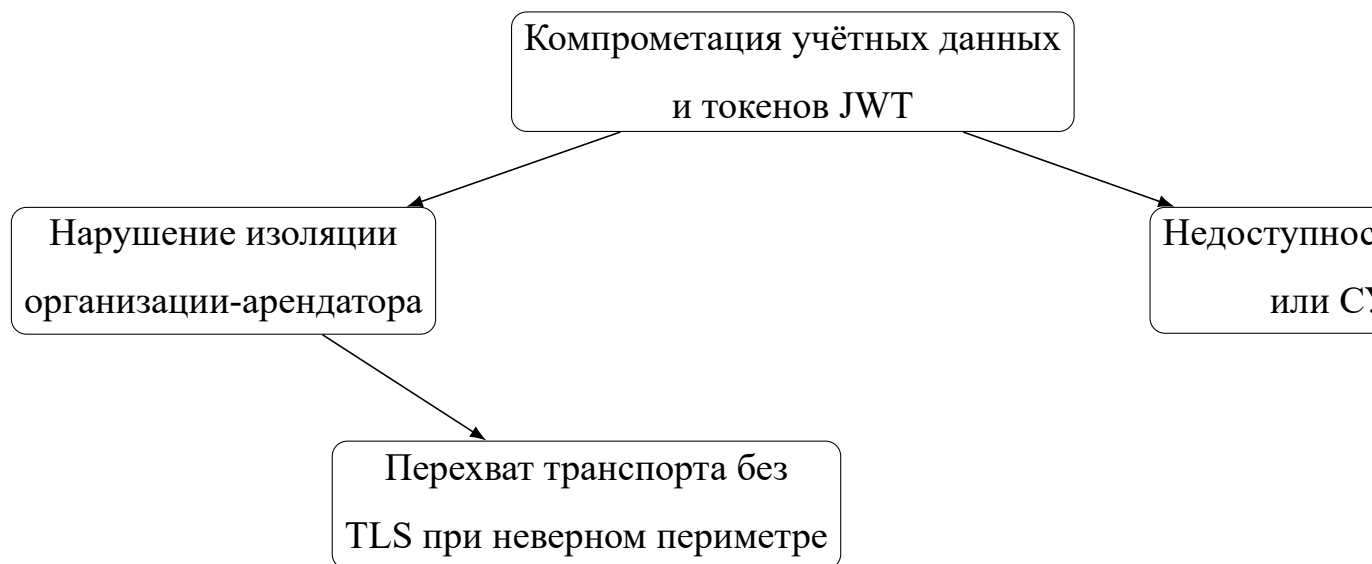


Рисунок 14 – Упрощённая схема классов угроз информационной безопасности для корпоративной системы управления обучением

3.3.2 Меры контроля, реализованные в коде

Реализованы хеширование паролей перед записью (`hash_password` в пакете `app.services.auth`) и выпуск JWT `access/refresh` после успешной аутентификации (`issue_tokens`, `rotate_refresh_token`). Обращения к ресурсам, зависящим от организации, защищены зависимостями `tenant_context`, проверкой активного членства и декораторами ограничения ролей `require_roles`.

Входные параметры фильтруются схемами `Rydantic`; для уменьшения риска инъекций утечного содержания HTML-содержание урока проходит очистку в сервисе `lesson_player`.

Ограничение частотных попыток аутентификации задается `auth_rate_limiter`; политика CORS настраивается списком `LMS_CORS_ORIGINS`.

В подсистеме аудита фиксируются ключевые операции записей (`write_audit_log`). Контекст организации в промышленной среде по `README` определяется поддоменом, в режиме разработки возможен допуск заголовка `X-Tenant-Code`.

Перечисленное согласовано со справочниками OWASP (в том числе ASVS базового уровня) и российскими базовыми требованиями к обработке персональных данных (Федеральный закон № 152-ФЗ), а качество свойств информационной системы — с ГОСТ Р ИСО/МЭК 25010[52, 54, 17, 13, 29, 26].

3.3.3 Замечания по усилению

Остаются рекомендации промышленного уровня: терминирование TLS на обратном прокси, веб-приложенческая стена приложений перед API, усиление ротации криптоматериалов, шифрование резервных копий тома данных, централизованная передача журналов действий оператора в систему мо-

мониторинга информационной безопасности (SOC). Реализации не предусмотрены в учебном стенде.

3.4 Надёжность, отказоустойчивость и производительность

Повышение наблюдаемости достигается наличием `/health` и `/health/db` и явной ошибкой недоступности СУБД на чтении.

Применение Alembic с привязкой схемы к ревизиям поддерживает переносимость развёртывания между средами[55, 30].

Контейнеризация обеспечивает повторяемость окружений; ошибки времени выполнения API возвращают типовые коды статусов HTTP через механизм FastAPI.

Текущие ограничения. Однопроцессный экземпляр серверной части без горизонтального масштабирования, одно инстансирующее узло PostgreSQL, отсутствие очередей для тяжёлых аналитических нагрузок.

Характеристики производительности. Без производственной нагрузки на демонстрационном наборе данных целевая производительность оценивается качественно: запрос списков в модулях курсов и тестирования воспринимается как оперативный; генерация следующего вопроса адаптивного теста определяется вычислительной работой фильтров и ограничителями сетевой связности между клиентом и сервером.

Развитие архитектуры. По мере промышленного масштабирования возможны добавление нескольких реплик серверной части за обратным балансатором нагрузки, репликация данных PostgreSQL, кеширование сессионных объектов средствами Redis или аналога, асинхронная обработка тяжёлых задач средствами брокера сообщений.

3.5 эксплуатация и развитие

Для образовательного результата и открытой публикации исходного кода программный комплекс рекомендуется распространять на условиях эксплуатации MIT как распространённого варианта для учебного программного проекта на стеке JavaScript/Python.

Авторское право на выпускную квалификационную работу охраняется нормами права Российской Федерации в части охраняемых выражений; при необходимости коммерциализации самостоятельного результата применимо свидетельство о государственной регистрации программы для ЭВМ.

развитие версий оформляется файлом журнала изменений CHANGELOG.md, выпусками тегированных релизов с политикой миграции схемы через Alembic и управлением уязвимостями зависимостей по CI.

3.6 Выводы по главе

В третьей главе выполнен анализ опытного внедрения программного комплекса в контейнерном контуре с использованием PostgreSQL 16, серверной части на FastAPI и веб-панели на порту 8081; отдельно описан запуск мобильного клиента на Flutter. Подтверждено закрытие пользовательского критического пути и соответствие ключевых требований по матрице проверок. Сформированы план и результаты интеграционного и функционального тестирования с отсылкой к приложению Б. Рассмотрены аспекты информационной безопасности с привязкой к реализованным механизмам аутентификации, изоляции организаций-арендаторов, политике CORS и аудиту. Зафиксированы ограничения по отказоустойчивости и перспективы масштабирования. Результаты подтверждают достижение цели ВКР и пригодность ком-

плекса для апробации в учебном процессе кафедры при дальнейшем развитии инфраструктурного контура.

ЗАКЛЮЧЕНИЕ

В результате выполнения выпускной квалификационной работы достигнута цель: разработан и описан программный комплекс корпоративного обучения с веб-панелью администрирования, мобильным клиентом слушателя и серверной частью REST API с поддержкой адаптивного тестирования и логической изоляции данных организаций-арендаторов.

Решены задачи, сформулированные во введении: выполнен анализ предметной области и существующих LMS; сформулированы и проверены требования; обоснован выбор технологий (FastAPI, SQLAlchemy, PostgreSQL/SQLite, React, TypeScript, Flutter, Docker Compose); спроектированы архитектура, модель данных и пользовательские сценарии; реализованы серверная, клиентская веб- и мобильная части; описаны алгоритмы адаптивного подбора вопросов, формирования рекомендаций и мультиарендной изоляции; приведены материалы опытного ввода в эксплуатацию и развёртывания, программа и результаты испытаний, анализ информационной безопасности и надёжность; подготовлены техническое задание, программа и методика испытаний, руководства пользователя, инструкция по развёртыванию и листинги кода в приложениях.

Практическая ценность работы состоит в том, что разработанный комплекс может служить основой для учебного стенда, пилотного внедрения в подразделении кафедры или организации и дальнейшего развития функциональности (SSO, расширение статистических моделей тестирования, геймификация). Дополнительно подтверждена работоспособность публичного контура: выполнено развёртывание на арендованном VPS с доменом coursum.online, доступ к REST API и выдача загруженных медиафайлов — через <https://api.coursum.online>. Научная новизна выражается в связке предметно-ориентированной модели корпоративного обучения с прозрачно

документированным алгоритмом дискретной адаптации сложности и механизмом рекомендаций по слабым темам в мультиарендной схеме данных.

По объёму пояснительная записка содержит введение, три главы, заключение, список использованных источников и пять приложений; иллюстрирована рисунками и таблицами; библиографический список включает 55 наименований в соответствии с требованиями нормоконтроля. Планируемая апробация — демонстрация на студенческой научной конференции и/или пилотное использование в учебном процессе кафедры по согласованию с руководителем.

Направления дальнейшей работы: внедрение полноценных моделей IRT (2PL/3PL) для адаптивного тестирования; интеграция с корпоративным единым входом (OIDC/SAML); развитие эксплуатации публичного VPS (резервное копирование, наблюдаемость, обновление зависимостей); расширение регрессионного и нагрузочного тестирования[1, 9, 53].

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Методические рекомендации по подготовке выпускных квалификационных работ (09.03.01, 09.03.03). – 2021.
2. Vocal Media. Mobile Learning Solutions in 2024: Trends and Innovations. – <https://vocal.media/education/mobile-learning-solutions-in-2024-trends-and-innovations>. – 2024. – Электронный ресурс. Дата обращения: 15.02.2026.
3. Opigno LMS. Finding the Perfect Mobile LMS App: Features Checklist. – <https://www.opigno.org/blog/finding-perfect-mobile-lms-app-features-checklist>. – 2024. – Электронный ресурс. Дата обращения: 15.02.2026.
4. Moodle US. A guide to LMS multi-tenancy: Do you need it? – <https://moodle.com/us/news/lms-multi-tenancy/>. – 2024. – Электронный ресурс. Дата обращения: 15.02.2026.
5. Yojji. Multi-Tenant LMS: The Complete Technical and Business Guide. – <https://yojji.io/blog/multi-tenant-lms>. – 2025. – Электронный ресурс. Дата обращения: 15.02.2026.
6. Грекул, Владимир Иванович. Проектирование информационных систем: учебное пособие / Владимир Иванович Грекул, Галина Николаевна Денищенко, Наталья Леонидовна Коровкина. – 3 edition. – Москва: ИД «ФОРУМ», 2022.
7. Куликов, С. С. Тестирование программного обеспечения. Базовый курс / С. С. Куликов. – 3-е изд. edition. – Минск: ЕРАМ Systems, 2024.
8. ГОСТ Р 7.0.5—2008. Система стандартов по информации, библиотечному и издательскому делу. Библиографическая ссылка. Общие требования и правила составления. – Москва: Стандартинформ, 2008.
9. ГОСТ 7.32—2017. Система стандартов по информации, библиотечному и издательскому делу. Отчёт о научно-исследовательской работе. Структура и правила оформления. – Москва: Стандартинформ, 2017.

10. ГОСТ 7.1—2003. Система стандартов по информации, библиотечному и издательскому делу. Библиографическая запись. Библиографическое описание. Общие требования и правила составления. – Москва: ИПК Издательство стандартов, 2003.

11. ГОСТ 2.105—2019. Единая система конструкторской документации. Общие требования к текстовым документам. – Москва: Стандартинформ, 2019.

12. ГОСТ 34.601—90. Информационная технология. Комплекс стандартов на автоматизированные системы. Автоматизированные системы. Стадии создания. – Москва: Издательство стандартов, 1990.

13. ГОСТ Р ИСО/МЭК 25010—2015. Информационные технологии. Системная и программная инженерия. Требования и оценка качества систем и программного обеспечения (SQuaRE). – Москва: Стандартинформ, 2015.

14. ГОСТ Р 50922—2006. Защита информации. Основные термины и определения. – Москва: Стандартинформ, 2006.

15. ГОСТ 19.201—78. Единая система программной документации. Техническое задание. Требования к содержанию и оформлению. – Москва: ИПК Издательство стандартов, 1978.

16. ГОСТ 19.301—79. Единая система программной документации. Программа и методика испытаний. Требования к содержанию и оформлению. – Москва: ИПК Издательство стандартов, 1979.

17. Федеральный закон от 27.07.2006 № 152-ФЗ «О персональных данных». – 2006. – Российская газета, Федеральный закон; редакция по состоянию на 2025 г.

18. Фаулер, Мартин. Архитектура корпоративных программных приложений / Мартин Фаулер. – Москва: Вильямс, 2021.

19. Дейт, Клайв Дж. Введение в системы баз данных / Клайв Дж. Дейт. – 8 edition. – Москва: Вильямс, 2020.

20. Коммервилл, Ян. Инженерия программного обеспечения / Ян Коммервилл. – 10 edition. – Москва: Вильямс, 2019.
21. Маклаков, Сергей Владимирович. Создание информационных систем с AllFusion Modeling Suite / Сергей Владимирович Маклаков. – Москва: ДМК Пресс, 2007.
22. Kaner, Cem. Lessons Learned in Software Testing: A Context-Driven Approach / Cem Kaner, James Bach, Bret Pettichord. – 1 edition. – New York: John Wiley & Sons, 2002.
23. Handbook of Modern Item Response Theory / Ed. by Wim J. van der Linden, Ronald K. Hambleton. – New York: Springer, 1997.
24. Computerized Adaptive Testing: A Primer / Howard Wainer, Neil J. Dorans, David Eignor et al. – 2 edition. – Mahwah, NJ: Lawrence Erlbaum Associates, 2000.
25. Lord, Frederic M. Applications of Item Response Theory to Practical Testing Problems / Frederic M. Lord. – Hillsdale, NJ: Lawrence Erlbaum Associates, 1980.
26. FastAPI Documentation. – <https://fastapi.tiangolo.com/>. – 2026. – Электронный ресурс. Дата обращения: 15.02.2026.
27. PostgreSQL Documentation. – <https://www.postgresql.org/docs/>. – 2026. – Электронный ресурс. Дата обращения: 15.02.2026.
28. Flutter Documentation. – <https://docs.flutter.dev/>. – 2026. – Электронный ресурс. Дата обращения: 15.02.2026.
29. Pydantic Documentation. – <https://docs.pydantic.dev/>. – 2026. – Электронный ресурс. Дата обращения: 08.05.2026.
30. SQLAlchemy Documentation. – <https://docs.sqlalchemy.org/>. – 2026. – Электронный ресурс. Дата обращения: 08.05.2026.
31. Moodle LMS. – <https://moodle.org/>. – 2026. – Электронный ресурс. Дата обращения: 15.02.2026.

32. Open LMS: The Open Source LMS That Fits Your Needs. – <https://www.openlms.net/>. – 2026. – Электронный ресурс. Дата обращения: 15.02.2026.
33. Moodle App. – <https://apps.apple.com/app/moodle/id633359593>. – 2026. – Электронный ресурс. Дата обращения: 15.02.2026.
34. MoodleCloud: Cloud LMS. – <https://moodlecloud.com/>. – 2026. – Электронный ресурс. Дата обращения: 15.02.2026.
35. Docebo Learning Network. Adaptive Learning: What It Is, Benefits & How It Works. – <https://www.docebo.com/learning-network/blog/adaptive-learning/>. – 2026. – Электронный ресурс. Дата обращения: 15.02.2026.
36. Feldstein, Michael. Adaptive Learning Systems: Surviving the Storm. – EDUCAUSE Review. – 2016. – Электронный ресурс. URL: <https://er.educause.edu/articles/2016/10/adaptive-learning-systems-surviving-the-storm>. Дата обращения: 15.02.2026.
37. Discovery Education. Intelligent Adaptive Learning. – <https://info.discoveryeducation.com/>. – 2025. – Электронный ресурс. Дата обращения: 15.02.2026.
38. Django REST Framework. – <https://www.django-rest-framework.org/>. – 2026. – Электронный ресурс. Дата обращения: 15.02.2026.
39. React: Documentation. – <https://react.dev/>. – 2026. – Электронный ресурс. Дата обращения: 15.02.2026.
40. TypeScript Documentation. – <https://www.typescriptlang.org/docs/>. – 2026. – Электронный ресурс. Дата обращения: 10.04.2026.
41. SQLite Documentation. – <https://sqlite.org/docs.html>. – 2026. – Электронный ресурс. Дата обращения: 10.04.2026.
42. Docker Compose Documentation. – <https://docs.docker.com/reference/compose-file/>. – 2026. – Электронный ресурс. Дата обращения: 10.04.2026.
43. Мартин, Роберт. Чистая архитектура. Искусство разработки программного обеспечения / Роберт Мартин. – СПб.: Питер, 2018.

44. Буч, Гради. Язык UML. Руководство пользователя / Гради Буч, Джеймс Рамбо, Айвар Джекобсон. – 2 edition. – Москва: ДМК Пресс, 2006.
45. ISO/IEC/IEEE 29119-2:2021. Software and systems engineering — Software testing — Part 2: Test processes. – 2021. – Международный стандарт на бумажном носителе и через каналы ISO.
46. Степанов, Владимир Анатольевич. Проектирование баз данных: учебное пособие / Владимир Анатольевич Степанов. – Москва: Академия, 2021.
47. ProProfs Training. Multi-Tenant LMS Guide 2025: Architecture, Features, and Use Cases. – <https://www.propofstraining.com/blog/multi-tenant-lms/>. – 2025. – Электронный ресурс. Дата обращения: 15.02.2026.
48. Microsoft. Architect multitenant solutions on Azure. – <https://learn.microsoft.com/azure/architecture/guide/multitenant/overview>. – 2024. – Электронный ресурс. Дата обращения: 08.05.2026.
49. Купер, Алан. Об интерфейсе. Основы проектирования взаимодействия / Алан Купер, Роберт Райман, Дэвид Кронин. – 3 edition. – Москва: Вильямс, 2009.
50. Норман, Дональд. Дизайн привычных вещей / Дональд Норман. – 2 edition. – Москва: Манн, Иванов и Фербер, 2015.
51. Круг, Стив. Не заставляйте меня думать. Веб-юзабилити и здравый смысл / Стив Круг. – 3 edition. – Москва: Вильямс, 2014.
52. OWASP. OWASP Top Ten 2021. – <https://owasp.org/Top10/>. – 2021. – Электронный ресурс. Дата обращения: 08.05.2026.
53. Сазерленд, Джефф. Scrum. Революционный метод управления проектами / Джефф Сазерленд. – Москва: Манн, Иванов и Фербер, 2016.
54. OWASP. Application Security Verification Standard (ASVS). – <https://owasp.org/ASVS/>. – 2025. – Электронный ресурс. Дата обращения: 08.05.2026.

55. SQLAlchemy ORM. Alembic Migrations Documentation. – <https://alembic.sqlalchemy.org/>. – 2026. – Электронный ресурс. Дата обращения: 08.05.2026.

ПРИЛОЖЕНИЕ А. ТЕХНИЧЕСКОЕ ЗАДАНИЕ

Наименование программного изделия: программный комплекс корпоративного обучения с веб-панелью администрирования, мобильным клиентом слушателя и серверной частью REST API.

Обозначение: ПК «Coursum-LMS».

Основание для разработки: выпускная квалификационная работа по направлению 09.03.03 «Прикладная информатика», профиль «Корпоративные информационные системы».

А.1 Введение

Наименование темы разработки: «Веб и мобильное приложение для корпоративного обучения с поддержкой адаптивного тестирования».

Объект автоматизации: процессы организации корпоративного обучения, назначения учебных программ, контроля освоения материала и оценки результатов в мультиарендной среде[15, 16, 12].

А.2 Назначение программы

Программа предназначена для автоматизации функций корпоративной системы управления обучением: ведение пользователей и ролей в разрезе организаций-арендаторов; управление курсами, уроками и медиаконтентом; назначение обучения пользователям и группам; проведение адаптивного тестирования с изменением сложности заданий; формирование рекомендаций по повторению тем; предоставление аналитики по прогрессу и результатам.

А.3 Требования к программе

А.3.1 Функциональные требования

- 1) система должна обеспечивать регистрацию, аутентификацию и выдачу токенов доступа и обновления;
- 2) система должна поддерживать выбор текущей организации и проверку членства пользователя в организации;
- 3) система должна реализовать роли слушателя, преподавателя, администратора организации и системного администратора с разграничением прав;
- 4) система должна обеспечивать создание и редактирование курсов, уроков, тестов и вопросов с указанием сложности;
- 5) система должна поддерживать назначение курсов пользователям и учебным группам;
- 6) система должна обеспечивать запуск попытки тестирования, выдачу следующего вопроса с учётом текущей сложности, приём ответа и завершение попытки;
- 7) система должна формировать рекомендации по темам для повторного изучения после завершения попытки;
- 8) система должна предоставлять аналитические сводки по курсам и слушателям в веб-панели.

А.3.2 Требования к надёжности

- 1) при недоступности СУБД серверная часть должна возвращать диагностируемый код ошибки;
- 2) операции изменения данных должны выполняться в транзакциях с сохранением ссылочной целостности;
- 3) журнал аудита должен фиксировать ключевые действия пользователей.

А.3.3 Условия эксплуатации

Клиентские приложения функционируют под управлением ОС семейств Windows, Linux (веб-браузер), Android/iOS (мобильное приложение). Серверная часть — под управлением ОС Linux или Windows с поддержкой контейнеризации Docker.

А.3.4 Состав и параметры технических средств

Минимальная конфигурация сервера: 2 ядра CPU, 4 Гбайт ОЗУ, 20 Гбайт диска; СУБД PostgreSQL не ниже версии 15 в составе контейнера.

А.3.5 Информационная совместимость

Обмен данными между клиентами и серверной частью — по протоколу HTTPS, формат тел сообщений JSON, базовый префикс API /api/v1.

А.3.6 Программная совместимость

Серверная часть: интерпретатор Python 3, фреймворк FastAPI, ORM SQLAlchemy, миграции Alembic. Веб-клиент: среда выполнения JavaScript, библиотека React, язык TypeScript. Мобильный клиент: Flutter/Dart.

А.3.7 Маркировка и упаковка

Поставка осуществляется в виде архива исходного кода или образов контейнеров с сопроводительным файлом версии и журналом изменений.

А.3.8 Требования по безопасности

Хранение паролей — только в виде криптографического хэша; доступ к данным арендатора — только после проверки идентификатора организации и роли; использование TLS на границе контура эксплуатации.

А.3.9 Эксплуатационная документация

Комплект включает руководство пользователя мобильного приложения, руководство администратора веб-панели, описание развёртывания (runbook), программу и методику испытаний.

А.4 Техничко-экономические показатели

Ожидаемый эффект внедрения связан со сокращением ручных операций при назначении обучения и учёте результатов по сравнению с разрозненными средствами; количественная оценка выходит за рамки настоящей работы и может быть выполнена отдельно по экономическим методикам организации-заказчика при пилотной эксплуатации.

А.5 Стадии и этапы разработки

- 1) техническое задание и уточнение требований;
- 2) проектирование архитектуры и модели данных;
- 3) реализация серверной части и REST API;
- 4) реализация веб-панели и мобильного клиента;
- 5) испытания и подготовка документации.

А.6 Порядок контроля и приёмки

Приёмка включает проверку соответствия функциональным требованиям по сценариям приложения Б, проверку развёртывания по инструкции приложения Г, анализ результатов автоматизированных тестов и контрольный просмотр журналов ошибок при демонстрации критического пути пользователя.

ПРИЛОЖЕНИЕ Б. ПРОГРАММА И МЕТОДИКА ИСПЫТАНИЙ

Б.1 Объект испытаний

Объектом испытаний является программный комплекс корпоративного обучения: серверная часть на базе FastAPI, веб-панель на React и TypeScript, мобильное приложение на Flutter, СУБД PostgreSQL (или SQLite в режиме разработки), конфигурация Docker Compose.

Б.2 Цель испытаний

Целью испытаний является подтверждение соответствия реализации функциональным и нефункциональным требованиям, описанным в техническом задании (приложение А), на критическом пути пользователей и при типовых операциях администрирования.

Б.3 Условия проведения испытаний

Испытания проводятся на стенде, развёрнутом по инструкции приложения Г, с использованием демонстрационных учётных записей и заполненной демонстрационной базы (seed_demo).

Б.4 Средства испытаний

- 1) персональная ЭВМ администратора с браузером и доступом к веб-панели;
- 2) мобильное устройство или эмулятор с установленным клиентским приложением;

3) средства автоматизированного тестирования: Pytest (серверная часть), Vitest и Playwright (веб-панель), Flutter test (мобильный клиент) — в соответствии с документом docs/testing-strategy.md в каталоге проекта.

Б.5 Методы испытаний

Применяются функциональное тестирование по тест-кейсам, интеграционное тестирование API, обзорное тестирование пользовательского интерфейса, проверка ограничений доступа по ролям и идентификатору организации[45, 22, 16].

Б.6 Порядок проведения и отчётность

Фиксируются версия сборки, дата прогона, результаты каждого тест-кейса (пройден / не пройден), идентификатор дефекта при отказе. Итоговый отчёт включает сводную таблицу и список замечаний.

Б.7 Таблица тест-кейсов критического пути

Таблица 12 – Результаты испытаний по тест-кейсам критического пути

ID	Описание	Шаги и ожидаемый результат	Фактический результат	Статус
ТС-01	Регистрация пользователя	POST /auth/register, создание записи пользователя	создан	ОК
ТС-02	Вход и получение JWT	POST /auth/login, выдача access и refresh токенов	выданы	ОК
ТС-03	Выбор организации	POST /tenants/select, активное членство	организация выбрана	ОК
ТС-04	Создание курса	POST /courses, запись с tenant_id	курс создан	ОК
ТС-05	Создание урока	POST /lessons, связь с курсом	урок создан	ОК
ТС-06	Создание теста и вопросов	POST /tests, POST вопросов	тест доступен	ОК
ТС-07	Назначение курса	POST назначения группе или пользователю	назначение создано	ОК
ТС-08	Список курсов слушателя	GET курсов под ролью learner	список возвращён	ОК
ТС-09	Старт попытки теста	POST /tests/{id}/start	попытка создана	ОК
ТС-10	Следующий вопрос	GET /attempts/{id}/next-question	вопрос или завершение	ОК
ТС-11	Отправка ответа	POST /attempts/{id}/submit-answer	обратная связь по сложности	ОК
ТС-12	Завершение попытки	POST /attempts/{id}/finish	результат и рекомендации	ОК
ТС-13	Рекомендации слушателя	GET /recommendations/me	список тем	ОК
ТС-14	Аналитика дашборда	GET /analytics/dashboard	агрегаты	ОК
ТС-15	Изоляция арендаторов	запрос данных другого tenant_id под чужой организацией	отказ 403/404	ОК
ТС-16	Доступ без токена	запрос к защищённому эндпоинту	401	ОК
ТС-17	Загрузка медиа	POST /media с файлом	URL медиа	ОК
ТС-18	Прогресс урока	POST прогресса страницы урока	сохранено	ОК
ТС-19	Веб-панель вход	страница авторизации и переход в shell	отображение разделов	ОК
ТС-20	Мобильный поток теста	экран TestFlowScreen: старт, ответы, финиш	завершено без падений	ОК

Примечание: колонки «Фактический результат» и «Статус» содержат ориентировочные значения для демонстрационной сборки; при защите подставляются фактические результаты прогона.

ПРИЛОЖЕНИЕ В. РУКОВОДСТВО ПОЛЬЗОВАТЕЛЯ МОБИЛЬНОГО ПРИЛОЖЕНИЯ И РУКОВОДСТВО АДМИНИСТРАТОРА ВЕБ-ПАНЕЛИ

В.1 Назначение

Настоящее приложение описывает пошаговые действия слушателя в мобильном клиенте и администратора или преподавателя в веб-панели при работе со стендом, собранным по инструкции приложения Г [28, 39, 26].

Общая политика демонстрации. Пароли в демонстрационных аккаунтах выражены литерально исключительно в учебных целях; в эксплуатации заменять на управляемые секретные фразы.

В.2 Демонстрационные учётные записи

Таблица 13 – Демонстрационные учётные записи программного комплекса

Электронная почта	Пароль (демо)	Назначение роли в сценариях проверки
sysadmin@example.com	Password123!	высокоуровневое администрирование платформы
admin@acme.example.com	Password123!	администрирование организации-арендатора учебного кейса
teacher@acme.example.com	Password123!	подготовка курсов, тестирования и аналитический просмотр
learner1@acme.example.com	Password123!	прохождение уроков, адаптивного теста и получение рекомендаций

В.3 Руководство пользователя мобильного приложения

В.3.1 Подключение к серверу

- 1) Открыть проект каталогом mobile, выполнить flutter pub get.
- 2) При использовании эмулятора Android зафиксировать базовый URL API как http://10.0.2.2:8000/api/v1; при физическом устройстве — IP узла локальной сети с портом 8000.

3) Запуск: `выполнить flutter run --dart-define=API_BASE=http://10.0.2.2:8000/api/v1` на эмуляторе Android; на устройстве подставить IP узла сети вместо 10.0.2.2.

В.3.2 Вход слушателя

1) Открыть экран входа, ввести `learner1@acme.example.com` и `Password123!`.

2) После успешной аутентификации убедиться, что назначенные курсы отображаются в списках и доступны для открытия.

В.3.3 Обучение и адаптивный тест

1) Из списка курсов открыть назначенный курс, перейти к урокам, пролистать страницы содержания; при наличии видеоматериала задействовать стандартные элементы воспроизведения плеера.

2) Для контроля знаний открыть сценарий адаптивного теста (экран `TestFlowScreen`); подтвердить старт попытки, поочерёдно отвечать на вопросы, наблюдая возможное изменение уровня сложности между вопросами.

3) Завершить попытку, просмотреть текстовые рекомендации по темам после обработки ответов системой.

В.4 Руководство администратора организации или преподавателя (веб-панель)

В.4.1 Подключение к веб-службе

1) По умолчанию при контейнеризированном стенде панель открывается на `http://localhost:8081`.

2) Убедиться, что при сборке веб-панели задаётся корректный базовый URL серверной части с суффиксом /api/v1.

В.4.2 Проверочный сценарий администратора организации

- 1) Авторизация аккаунтом admin@acme.example.com.
- 2) Проверка раздела пользователей: назначить роли, при необходимости создать учебную группу для массового назначения.
- 3) Создание объекта учебной программы или проверка существования демонстрационной программы после сидирования.

В.4.3 Проверочный сценарий преподавателя

- 1) Авторизация аккаунтом teacher@acme.example.com.
- 2) В разделе «Тесты» подготовить тестируемый объект или отредактировать существующий: поля ограничителя попытки, активный тест, банк вопросов.
- 3) Назначить курс аудитории слушателей через соответствующий раздел.
- 4) Открыть аналитический раздел панели (сводка по активности обучения) и проверить отображение агрегированных результатов после прохождения тестирования слушателем.

В.4.4 Системная роль платформы (по необходимости)

Аккаунт sysadmin@example.com служит контрольным наблюдением за экземпляром при многофирменной модели данных; использовать только на защите по сценарию, согласованному с методическим указанием.

В.5 Известные ограничения демонстрации

При работе стенда только в локальной сети необходимо, чтобы браузер и мобильный клиент разрешали обращение по протоколу HTTP к адресам узла сборки; в производственном контуре использовать обратное проксирование с TLS.

ПРИЛОЖЕНИЕ Г. РУКОВОДСТВО ПО РАЗВЁРТЫВАНИЮ ПРОГРАММНОГО КОМПЛЕКСА

Г.1 Подготовка среды

- 1) Установить Docker Engine версии актуальной ветви и утилиту Docker Compose (docker compose).
- 2) По необходимости установить среду сборки Flutter и Android SDK или Xcode для сборки мобильного приложения слушателя.
- 3) Клонировать исходный код по адресу <https://github.com/lunahobi/coursem>.

Г.2 Файлы конфигурации

- 1) Скопировать .env.example из корня проекта (или каталога серверной части) во внутреннее имя .env согласно путям сборки Dockerfile.
- 2) Зафиксировать минимально: POSTGRES_PASSWORD, LMS_SECRET_KEY достаточной длины, LMS_ENVIRONMENT, LMS_CORS_ORIGINS в форме JSON-массива или перечне разрешённых источников.
- 3) Для режима локальной диагностики допускать LMS_ALLOW_TENANT_HEADER_FALLBACK=true для передачи кода организации заголовком X-Tenant-Code согласно реализации серверной части.

Г.3 Запуск через Docker Compose

- 1) В корне каталога с исходным кодом выполнить docker compose up --build -d.

2) Дождаться готовности службы базы данных: контейнер db использует postgres:16-alpine и публикуется на 127.0.0.1:5433.

3) После выхода сценария entrypoint успешности миграций (Alembic) сервер слушает 127.0.0.1:8000, сборка статики веб-панели экспонируется на 127.0.0.1:8081.

Г.4 Миграции схемы базы данных

Миграции запускает entrypoint-последовательность контейнерной серверной части после проверки согласования существования таблицы alembic_version.

При ручном запуске серверной части в каталоге backend допускается alembic upgrade head при активированном виртуальном окружении интерпретатора Python 3[55, 42].

Г.5 Первоначальные данные

1) Для профиля tools контейнерного набора выполнить docker compose run --rm --profile tools seed.

Альтернативно локально в каталоге backend при готовности переменной LMS_DATABASE_URL: python -m scripts.seed_demo.

2) Проверить подключением к БД наличие демонстрационных ключей записей пользователей.

Г.6 Проверки готовности

1) Открыть GET http://localhost:8000/health.

2) Открыть GET http://localhost:8000/health/db.

3) Обращаться к наблюдаемому REST интерфейсу приложения через префикс /api/v1.

Г.7 Мобильное приложение

1) Собрать в каталоге mobile командой `flutter run` с параметром `--dart-define=API_BASE=....`

2) Для эмулятора Android использовать `http://10.0.2.2:8000/api/v1`; для устройства в LAN — базовое имя узла сборки компьютера-разработчика.

Г.8 Остановка стенда

Для сохранности данных достаточно `docker compose stop`.

Для полного удаления контейнеров и томов PostgreSQL использовать `docker compose down -v`.

Г.9 Пример опытного развёртывания на VPS (домен **coursum.online**)

В рамках демонстрации работоспособности программного комплекса в сети Интернет использовался арендованный виртуальный выделенный сервер (VPS). Зарегистрировано доменное имя **coursum.online**; для пользовательского веб-приложения и мобильного клиента базовый URL API в конфигурации сборки задаётся на узел `https://api.coursum.online/api/v1`. Выдача загруженных медиафайлов пользователям и слушателям осуществляется через тот же хост **api.coursum.online**, так как отдача каталога `/media` настроена в серверной части наряду с REST API.

Согласование источников для CORS и базового URL веб-панели задаётся переменными окружения (в проекте задано: `LMS_CORS_ORIGINS` включает `https://coursum.online` и `https://www.coursum.online`; параметр сборки веб-клиента `VITE_API_BASE` указывает на `https://api.coursum.online/api/v1`). Развёртывание на VPS выполнялось в контейнерном виде командой `docker compose up -d --build` после настройки DNS-записей на IP-адрес сервера и

выпуска TLS-сертификата (например, средствами обратного прокси с поддержкой ACME).

ПРИЛОЖЕНИЕ Д. ЛИСТИНГИ КЛЮЧЕВОГО КОДА

Ниже приведены иллюстративные фрагменты исходных текстов программного комплекса; полный набор файлов и история изменений находятся в репозитории <https://github.com/lunahobi/coursum>. Фрагменты сокращены для читаемости, секретные константы не воспроизводятся. Нумерация листингов относится к настоящему приложению [26, 28, 29, 42].

Д.1 Алгоритм адаптивного теста (дискретное изменение сложности ответом)

Листинг Д.1 – Фрагмент `evaluate_answer` (backend/app/services/adaptive.py)

```
1  def evaluate_answer(  
2      db: Session,  
3      *,  
4      attempt: Attempt,  
5      question: Question,  
6      answer_option_id: int | None,  
7      response_seconds: int,  
8  ) -> dict:  
9      options = db.scalars(select(AnswerOption).where(AnswerOption.question_id ==  
10         question.id)).all()  
11      option_ids = {option.id for option in options}  
12      if answer_option_id is not None and answer_option_id not in option_ids:  
13          raise HTTPException(status_code=422, detail='Answer option does not belong to  
14             question')  
15      correct_option = next((opt for opt in options if opt.is_correct), None)  
16      is_correct = bool(correct_option and correct_option.id == answer_option_id)  
17      previous_difficulty = attempt.current_difficulty  
18      attempt.asked_question_ids = [(attempt.asked_question_ids or []), question.id]  
19      attempt.current_difficulty = clamp_difficulty(attempt.current_difficulty + (1 if  
20         is_correct else -1))  
21      attempt.difficulty_path = [(attempt.difficulty_path or []), attempt.  
22         current_difficulty]  
23      answer = AttemptAnswer(  
24          tenant_id=attempt.tenant_id,  
25          attempt_id=attempt.id,
```



```

22         question_id=question.id,
23         answer_option_id=answer_option_id,
24         is_correct=is_correct,
25         response_seconds=response_seconds,
26     )
27     db.add(answer)
28     # ... итог возвращается в виде словаря с ключами is_correct, current_difficulty
    ...

```

Д.2 Ограничение HTML содержания урока при проигрывании

Листинг Д.2 — Разрешённые теги и протоколы
(backend/app/services/lesson_player.py)

```

1  ALLOWED_HTML_TAGS = [
2      'p', 'br', 'div', 'span', 'strong', 'em', 'b', 'i', 'u', 'mark',
3      'small', 'code', 'pre', 'blockquote', 'hr', 'ul', 'ol', 'li',
4      'h1', 'h2', 'h3', 'h4', 'table', 'thead', 'tbody', 'tr', 'th',
5      'td', 'a', 'img', 'video', 'source',
6  ]
7  ALLOWED_PROTOCOLS = ['http', 'https']

```

Д.3 Регистрация и выпуск JWT

Листинг Д.3 — Обработка учётной записи и токены
(backend/app/services/auth.py)

```

1  def register_user(db: Session, *, email: str, full_name: str, password: str) -> User:
2      if db.scalar(select(User).where(User.email == email)):
3          raise HTTPException(status_code=status.HTTP_400_BAD_REQUEST, detail='Email
         already registered')
4      user = User(email=email, full_name=full_name, password_hash=hash_password(
         password))
5      db.add(user)
6      db.flush()
7      # ... запись в журнал аудита и возврат user опущены ...
8
9  def issue_tokens(db: Session, user: User) -> tuple[str, str]:
10     access = create_token(str(user.id), 'access', settings.access_token_minutes)

```

```

11     refresh = create_token(str(user.id), 'refresh', settings.refresh_token_minutes)
12     db.add(
13         RefreshToken(
14             user_id=user.id,
15             token=refresh,
16             expires_at=utcnow() + timedelta(minutes=settings.refresh_token_minutes),
17             revoked=False,
18         )
19     )
20     return access, refresh

```

Д.4 Обработчик старта попытки на маршруте тестирования

Листинг Д.4 – Старт попытки (backend/app/api/routes/tests.py)

```

1  @router.post('/tests/{test_id}/start')
2  def start_test(
3      test_id: int,
4      membership: Membership = Depends(active_membership),
5      tenant: Tenant = Depends(tenant_context),
6      db: Session = Depends(get_db),
7  ) -> dict:
8      test = db.scalar(select(Test).where(Test.id == test_id, Test.tenant_id == tenant.
9          id))
10     if test is None:
11         raise HTTPException(status_code=404, detail='Test not found')
12     question_count = db.scalar(
13         select(func.count(Question.id)).where(
14             Question.test_id == test.id,
15             Question.tenant_id == tenant.id,
16         )
17     ) or 0
18     if question_count == 0:
19         raise HTTPException(status_code=400, detail='Test has no questions')
20     attempt = Attempt(
21         tenant_id=tenant.id,
22         test_id=test.id,
23         user_id=membership.user_id,
24         current_difficulty=test.baseline_difficulty,
25         asked_question_ids=[],

```

```

25         difficulty_path=[test.baseline_difficulty],
26     )
27     db.add(attempt)
28     db.commit()
29     db.refresh(attempt)
30     return {'attempt_id': attempt.id}

```

Д.5 Ключевые сущности ролевой модели данных

Листинг Д.5 — Роли пользователя и организация
(backend/app/models/models.py)

```

1  class RoleName(StrEnum):
2      learner = 'learner'
3      teacher = 'teacher'
4      org_admin = 'org_admin'
5      system_admin = 'system_admin'
6
7  class Tenant(Base):
8      __tablename__ = 'tenants'
9
10 class User(Base):
11     __tablename__ = 'users'
12     email: Mapped[str]
13     full_name: Mapped[str]
14     password_hash: Mapped[str]

```

Д.6 Веб-панель: редактор тестирования преподавателя

Листинг Д.6 — Компонент TestsPage (web/src/app-core.tsx, фрагмент)

```

1  export function TestsPage({ session }: { session: SessionState }) {
2      const { language, t } = useUi();
3      const courses = useRemote<CourseInfo[]>('/courses', session);
4      const tests = useRemote<TestInfo[]>('/tests', session);
5      const [refreshKey, setRefreshKey] = useState(0);
6      // ...
7      async function submit(event: FormEvent) {
8          event.preventDefault();

```

```

9      const created = (await apiPost('/tests', session, {
10        course_id: Number(form.courseId),
11        title: form.title,
12        baseline_difficulty: form.baselineDifficulty,
13        question_limit: form.questionLimit,
14      }))) as { id?: number };
15      // ...
16    }
17  }

```

Д.7 Мобильное приложение: экран попытки

Листинг Д.7 – TestFlowScreen (mobile/lib/main.dart, фрагмент)

```

1  class TestFlowScreen extends StatefulWidget {
2    const TestFlowScreen({
3      super.key,
4      required this.api,
5      required this.session,
6      required this.testId,
7    });
8
9    final ApiClient api;
10   final SessionState session;
11   final int testId;
12
13   @override
14   State<TestFlowScreen> createState() => _TestFlowScreenState();
15 }
16
17 class _TestFlowScreenState extends State<TestFlowScreen> {
18   Future<void> start() async {
19     final started = await widget.api.startAttempt(widget.testId, widget.session);
20     attemptId = started['attempt_id'] as int;
21     await loadNextQuestion();
22   }
23 }

```

Д.8 Оркестрация контейнеров демонстрационного контура

Листинг Д.8 – Сервисы docker-compose.yml, фрагмент

```
1  services:
2    db:
3      image: postgres:16-alpine
4      ports:
5        - "127.0.0.1:5433:5432"
6    backend:
7      build:
8        context: ./backend
9      ports:
10       - "127.0.0.1:8000:8000"
11   web:
12     build:
13       context: ./web
14     ports:
15       - "127.0.0.1:8081:80"
```

Примечание: в учебном стенде не приводится полное содержимое файлов конфигурации и переменные окружения с секретными значениями.